DFT设计

DESIGN FOR TESTABILITY

BZ

CONTENTS

- 测试目标 — DFT 测什么？三类对象各用什么手段？
- 测试原理 — 每种方法怎样建立通路、控制和观察？
- 工程流程 — 各方法在工程流程中负责哪一步？
- 附录 — 哪些做法本流程只点名、不作主路径？
- 常见错误与排查 — 报错了怎么读、怎么修、去手册哪里找例子？

01

Part 1　测试目标

三类被测对象与三类测试手段

DFT 测什么？三类对象各用什么手段？

DFT：让 ATE 经少数管脚检出芯片内部的制造缺陷

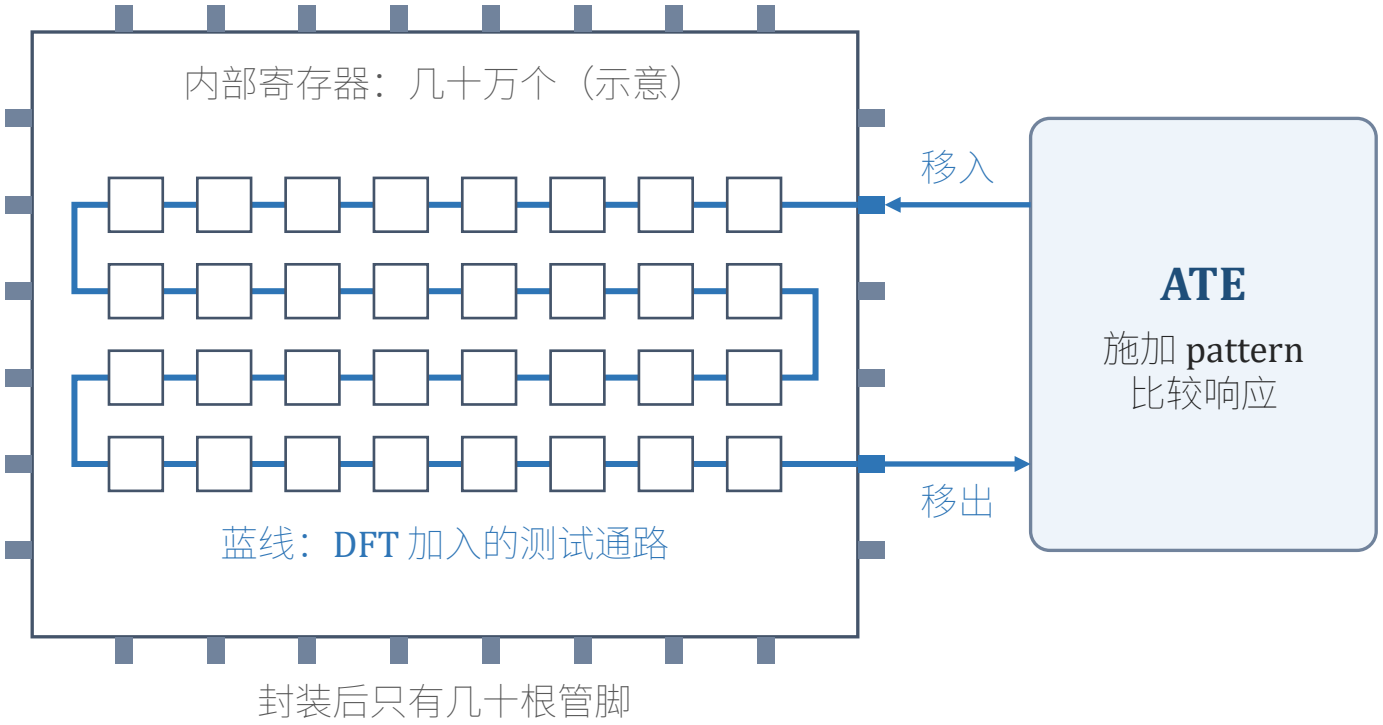
DFT 在设计中加入测试通路并生成测试 pattern，让 ATE 在出厂前检出制造缺陷。

质量：在出厂前检出制造缺陷

效率：压缩测试时间与数据量

诊断：把失败定位到具体结构

边界：测制造缺陷，不验功能

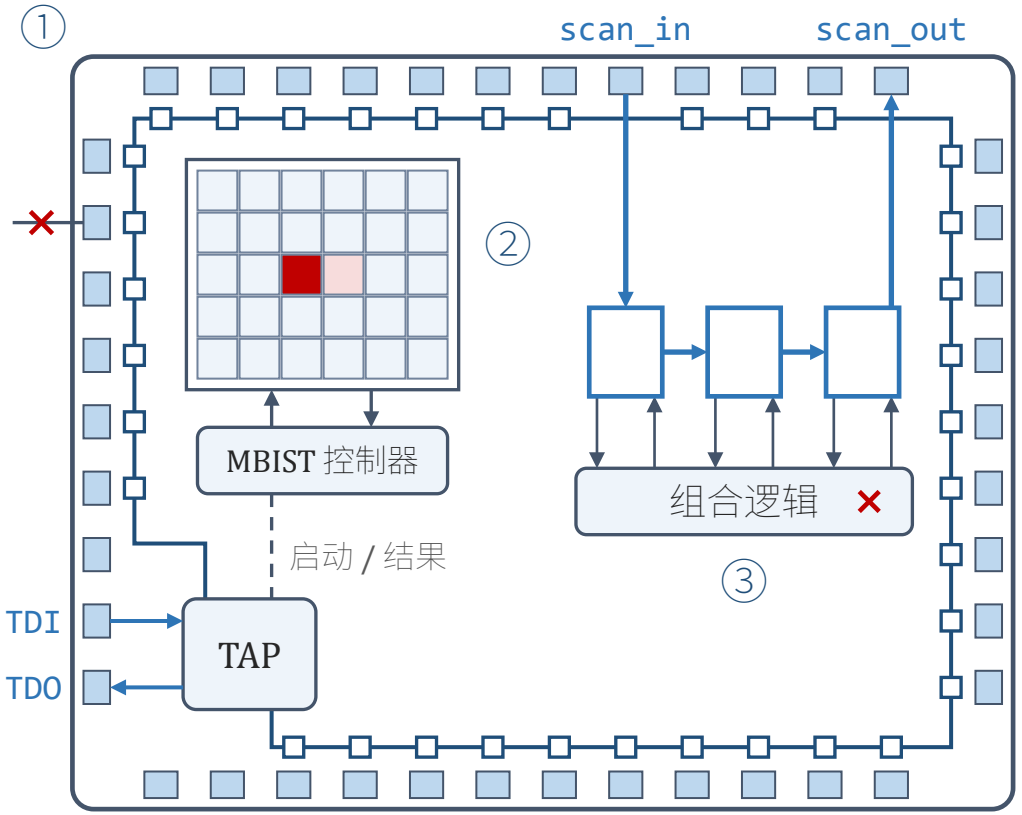


ATE：Automated Test Equipment（自动测试设备）

→ 三类被测对象各是什么？

三类被测对象：引脚互连、嵌入式存储器、随机逻辑

三类对象各有一种专门的测试：加的硬件、放的位置、数据走的路，都不一样。



① 引脚与板级互连：边界扫描

故障：开路、短路

加：管脚处的边界单元，串成边界寄存器

数据：`TDI` → 边界寄存器 → `TDO`，经 TAP

② 嵌入式存储器：**MBIST**

故障：单元 stuck-at、耦合、地址译码

加：存储器旁的 MBIST 控制器

数据：片上产生、写读、比较；TAP 只启动、读结果

③ 随机逻辑：扫描链 **+ ATPG**

故障：门与寄存器的结构缺陷

加：触发器换成扫描单元，串成扫描链

数据：`scan_in` → 扫描链 → `scan_out`，ATE 比较

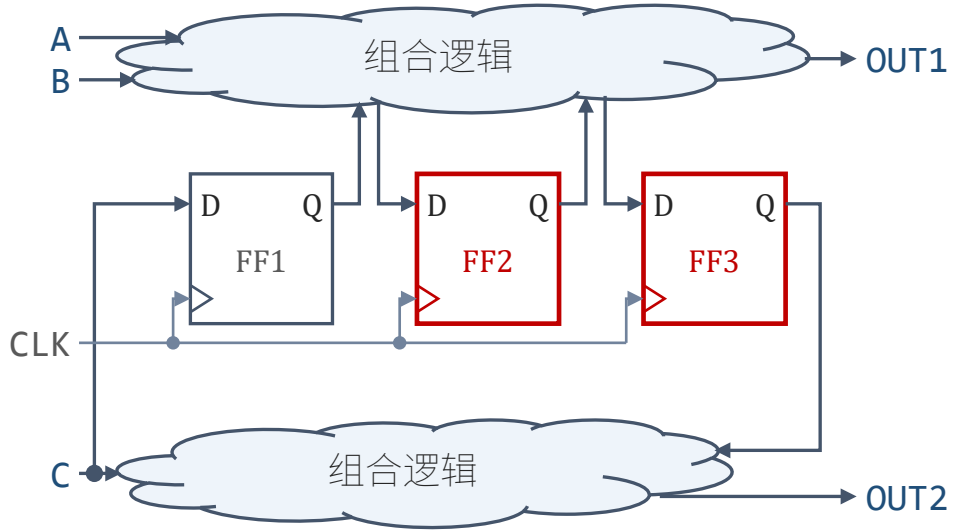
TAP：Test Access Port；MBIST：Memory Built-In Self-Test；ATPG：Automatic Test Pattern Generation

→ 检出一个故障需要哪两个条件？

检出故障的两个条件：可控与可观

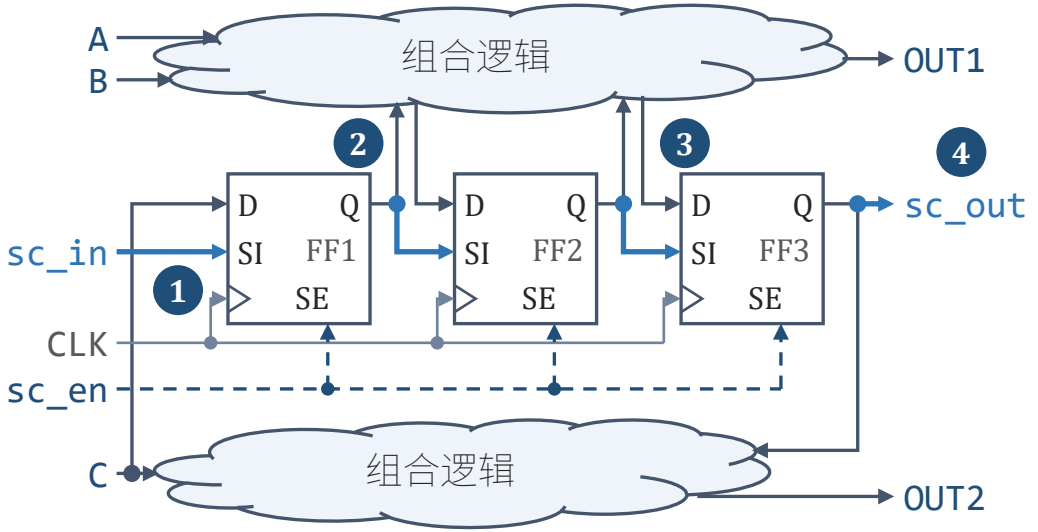
可控：能把目标节点设成所需值；可观：能把差异传到 ATE 可比较的输出。

加扫描前：输入 **A**、**B**、**C**，输出 **OUT1**、**OUT2**



红色 FF2、FF3：设值、读值都要穿过组合逻辑

加扫描后：多出 **sc_in**、**sc_en**、**sc_out**



蓝线：扫描链绕过组合逻辑；虚线：sc_en；细线：CLK

① 移位：sc_en = 1，已知值经 sc_in 移入各 FF

② 可控：FF 的 Q 直接驱动组合逻辑

③ 捕获：sc_en = 0，打一拍 CLK，响应装进 FF

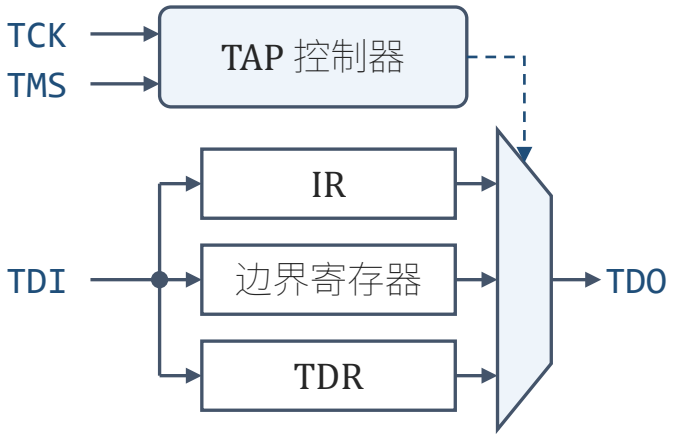
④ 可观：再移位，响应经 sc_out 送 ATE 比较

→ 三类对象各用什么手段提高可控与可观？

三类手段：**access**、扫描 **+ ATPG**、内建自测试（**BIST**）

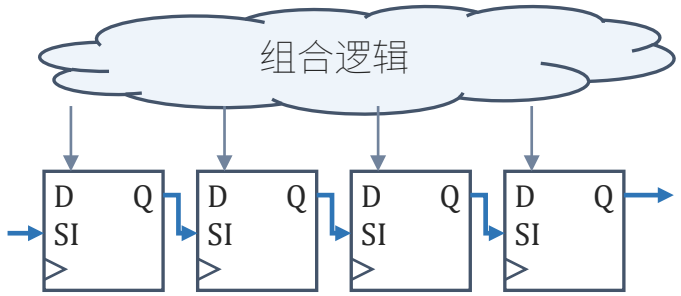
三类手段恰好对着三种测试：边界扫描靠 access，扫描链靠扫描 + ATPG，MBIST 属于 BIST。

access



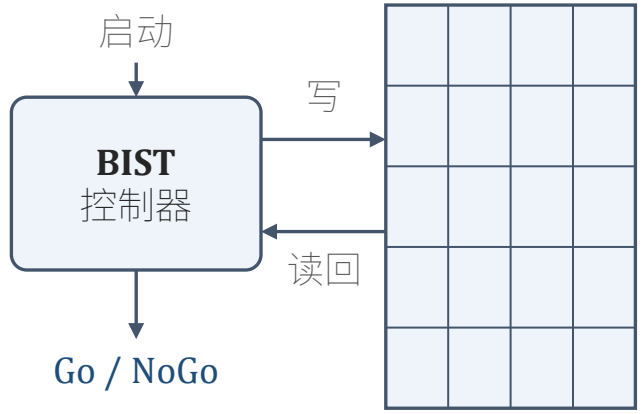
IR：选定 TDI 到 TDO 的寄存器
边界寄存器：串接各 I/O
TDR：驱动片上 instrument

扫描 + ATPG



寄存器串成扫描链
移入 pattern，捕获一拍
移出响应，由 ATE 比较

内建自测试



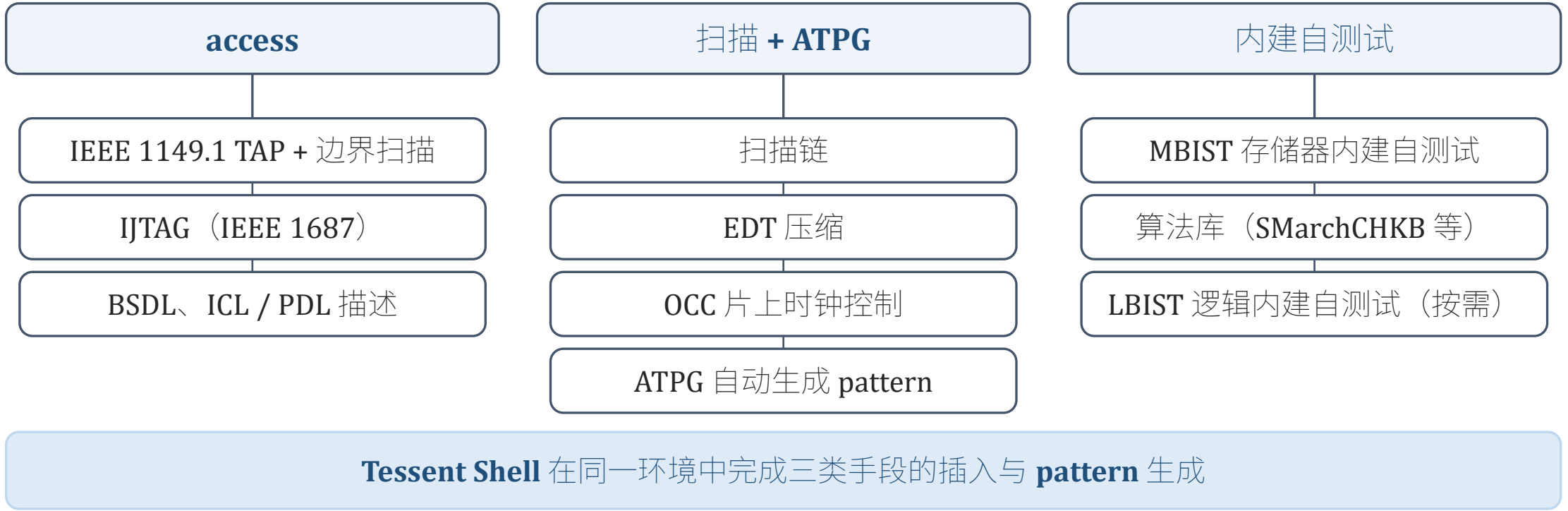
控制器在片上产生 pattern
读回数据与期望值比较
外部只启动、读取结果

IR：Instruction Register；TDR：Test Data Register；instrument：片上测试电路，如 MBIST 控制器；BIST：Built-In Self-Test

→ 三类手段在工业上怎样实现？

三类手段的工业实现：**IEEE** 标准与 **Tessent** 工具

access 遵循 IEEE 1149.1 / 1687；扫描、压缩与 BIST 由 EDA 工具插入并生成 pattern。

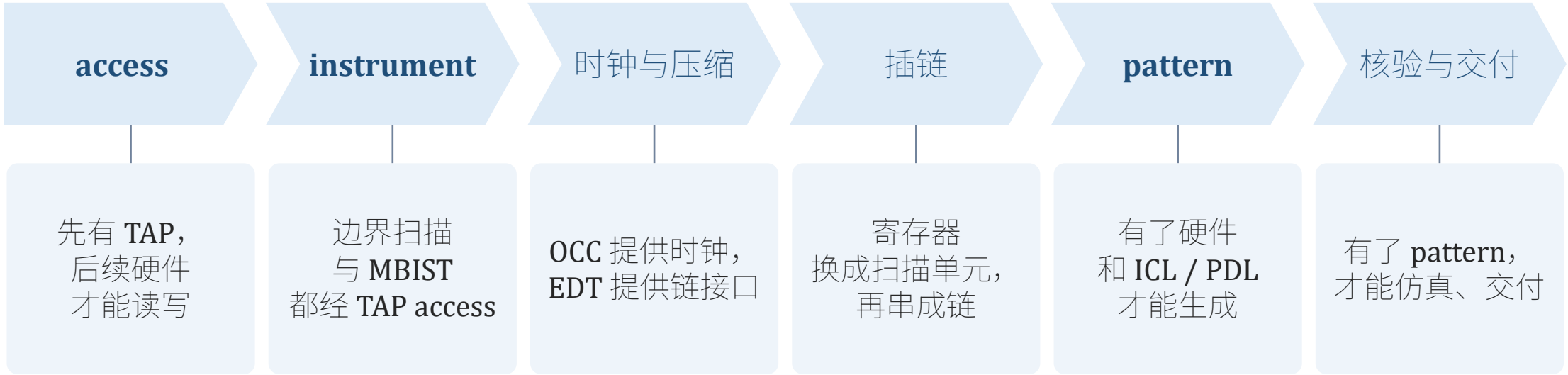


IJTAG：Internal Joint Test Action Group；EDT：Embedded Deterministic Test；OCC：On-Chip Clock Controller

→ 硬件与 pattern 按什么先后顺序完成?

DFT 流程的顺序由硬件依赖决定

测试硬件与 access 描述（ICL / PDL）齐备后，才能生成 pattern、仿真和交付。



没有扫描链就没有 ATPG；没有 ICL / PDL，就生成不了 instrument 的 pattern。

instrument：接在 test access 网络上的片上测试电路，如 MBIST 控制器、OCC；ICL / PDL：其连接与操作描述

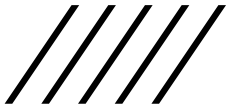
→ Part 1 的要点是什么？

Part 1 小结：用测试通路让缺陷可控、可观、可诊断

Part 1 结束时，应能说出这四句话：

目标	管脚很少时，仍能控制、观察芯片内部的制造缺陷
对象	引脚互连（边界扫描）、嵌入式存储器（MBIST）、随机逻辑（扫描链 + ATPG）
手段	access（TAP / IJTAG）、扫描 + ATPG（配 EDT、OCC）、内建自测试（MBIST）
流程	依赖顺序：access → instrument → 时钟与压缩 → 插链 → pattern → 核验交付

→ Part 2：每种方法怎样建立通路、控制和观察？



02

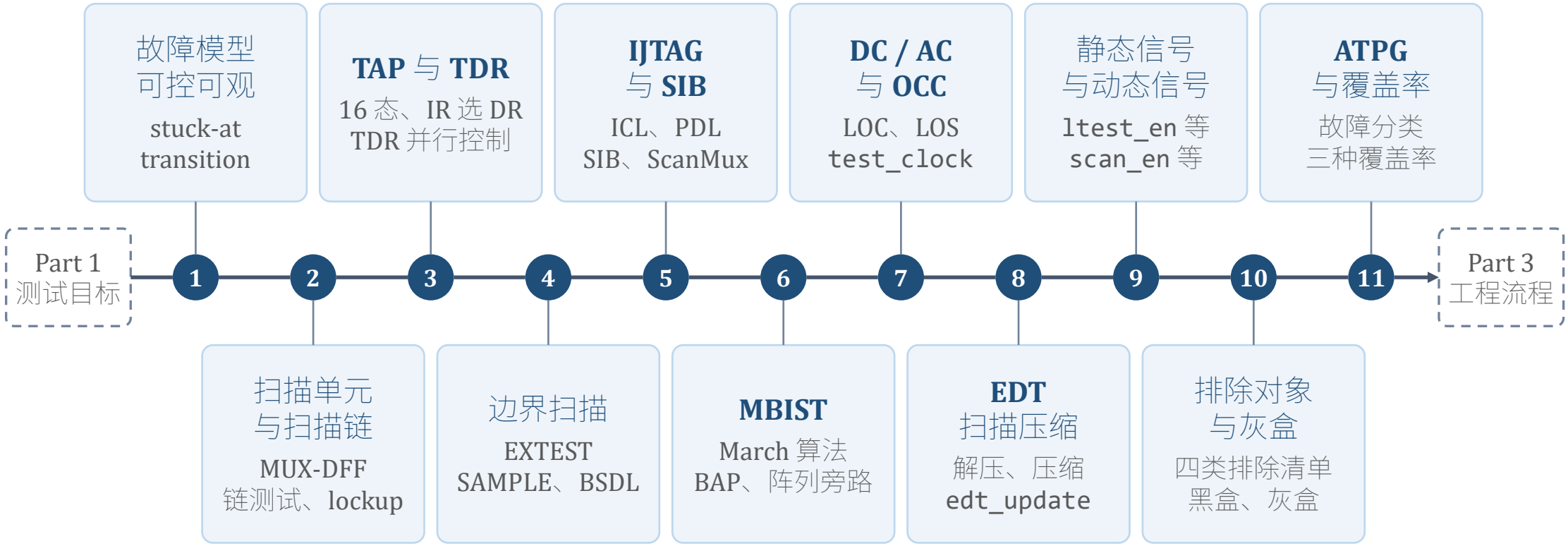
Part 2　测试原理

access、扫描、存储器、时钟与压缩、覆盖率

每种方法怎样建立通路、控制和观察?

Part 2 路线图：从故障模型到覆盖率，11 个主题

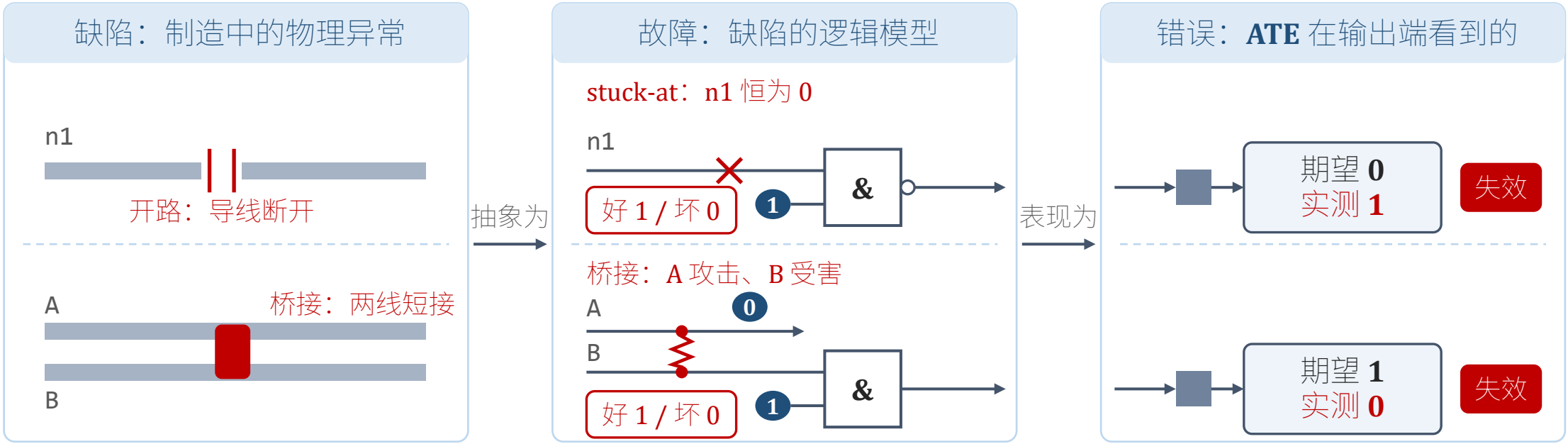
从故障模型与扫描基础出发，逐步进入 test access、instrument、压缩和覆盖率分析。



→ 缺陷、故障、错误有什么区别?

缺陷、故障、错误：**pattern** 按故障模型生成

硅上的缺陷先抽象成故障模型，才能按模型生成 pattern；ATE 看到的是错误值。



故障模型：**stuck-at** **transition** 桥接 **IDDQ** 路径延时　深色为本课主用

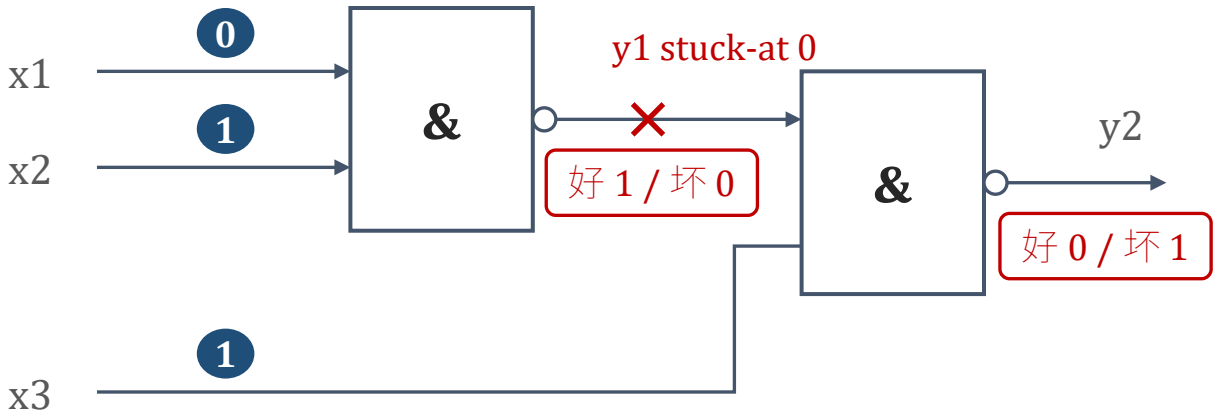
开路常按 stuck-at 建模；桥接要让两线取相反值，由攻击线改写受害线；后文主要用 stuck-at 与 transition 模型。

→ stuck-at 故障：怎样激活，又怎样传播？

stuck-at 故障：先激活，再传播

stuck-at 故障把节点视为恒为 0 或恒为 1，下例目标为 y1 stuck-at 0。

例：两级与非门



① 激活：把故障点设成相反的值

目标 y1 stuck-at 0：好电路要让 y1 = 1

取 x1 = 0（或 x2 = 0）即可

② 传播：把差异送到可观测点

差异要经后级才能到达 y2

x3 = 1：后级与非门把差异传到 y2

→ transition 故障：为什么慢速测试测不到？

transition 故障：慢速测试测不到

transition 故障：节点能到 0 和 1，却不能在规定时间内完成翻转。

两个向量：V1 初始化，V2 发射

慢速测试：时间充裕，翻转总能完成

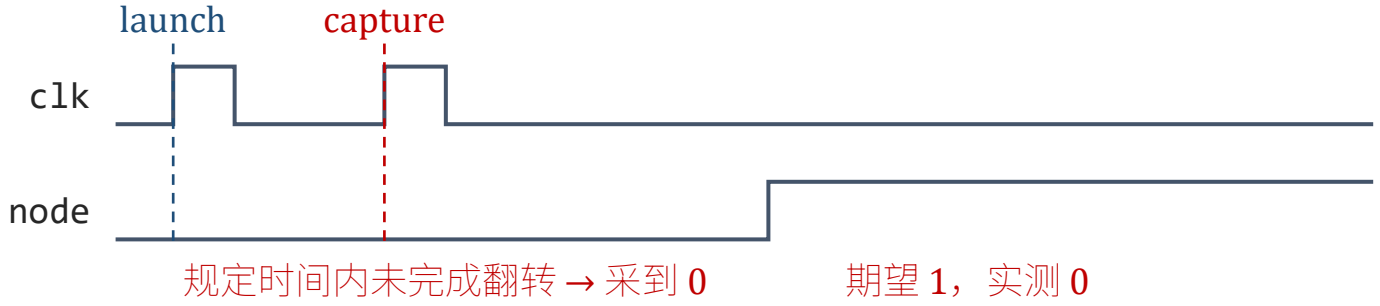
慢速捕获：看不出故障



至速捕获：两拍快钟由 OCC 产生

两个方向：slow-to-rise / slow-to-fall

至速捕获：采到旧值

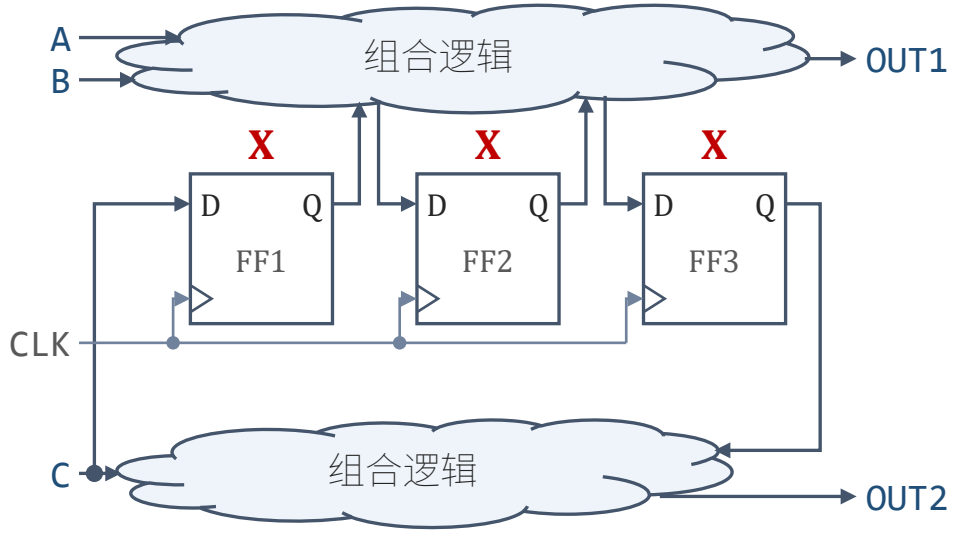


→ 激活失败与传播失败在报告里怎样记?

激活不足记为 **UC**，传播不到记为 **UO**

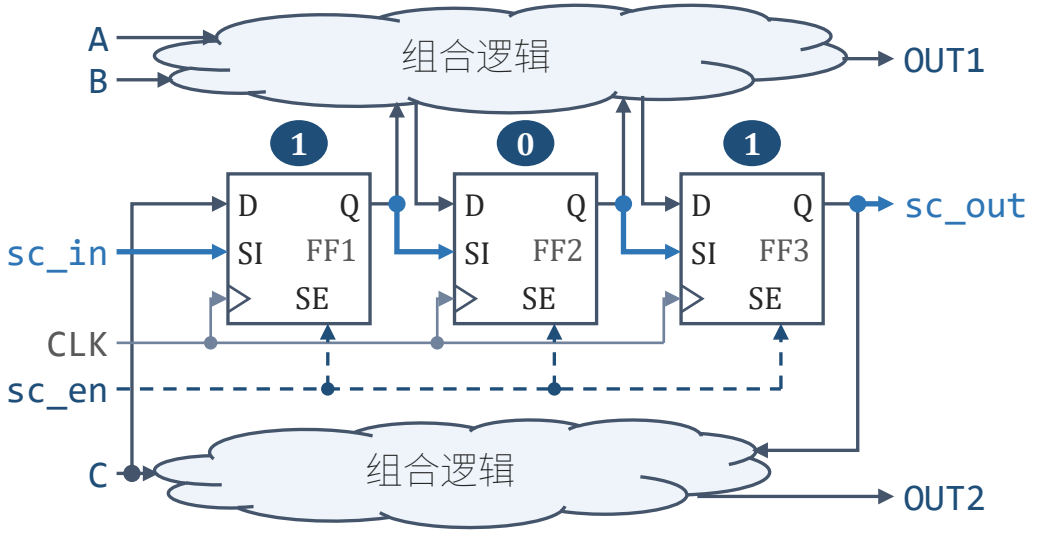
无扫描时内部状态是未知的 **X**：目标点设不成所需值，差异也传不到输出。

无扫描：内部状态是 **X**



X：上电后状态未知，经管脚也难以设定

有扫描：已知 **0 / 1** 移入，响应移出



可控：从 `sc_in` 设值；可观：从 `sc_out` 读响应

不可控 → 激活失败：常记为 UC　　不可观 → 传播失败：常记为 UO　　补法：加扫描、测试点或控制位

UC：Uncontrolled；UO：Unobserved

→ 扫描单元怎样在功能与移位之间切换？

扫描单元 MUX-DFF：scan_en 选功能数据或移位数据

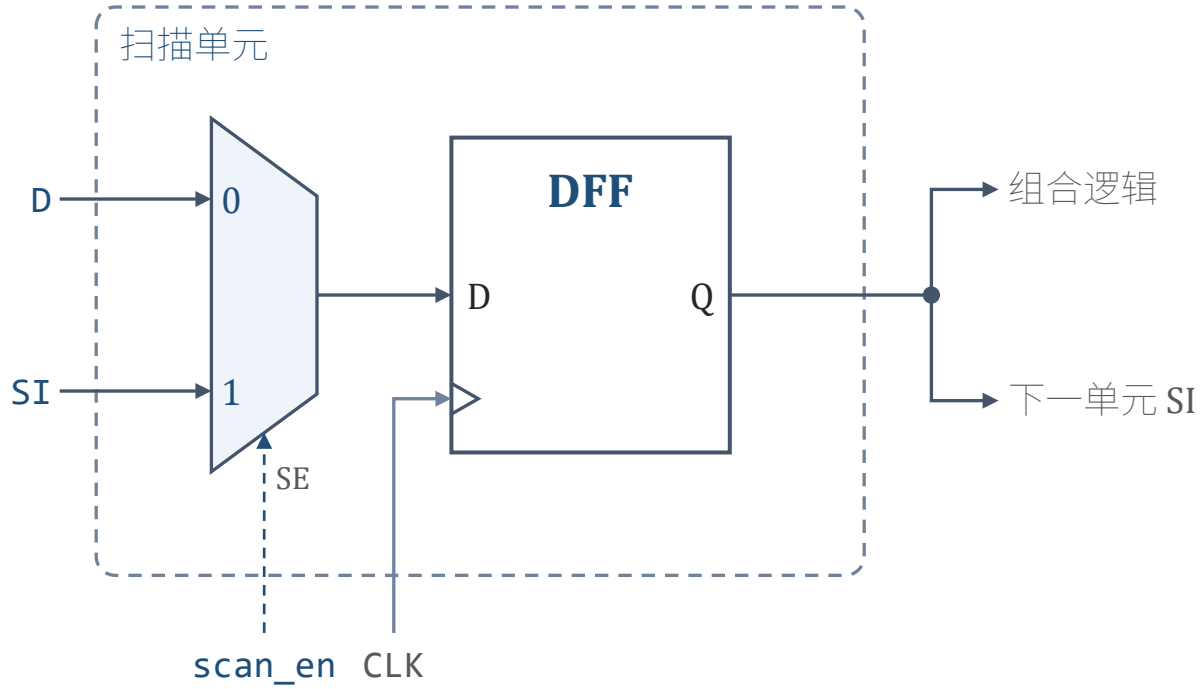
扫描单元 = 二选一 MUX + DFF：同一个触发器既能捕获功能数据，也能串成移位链。

结构：DFF 的 D 端前加二选一 MUX

连接：Q 驱动逻辑，也接下一单元 SI

类型：Tessent 支持 MUX-DFF、LSSD 及两者混合

scan_en	FF 装入	含义
0	D	功能 / 捕获
1	SI	移位

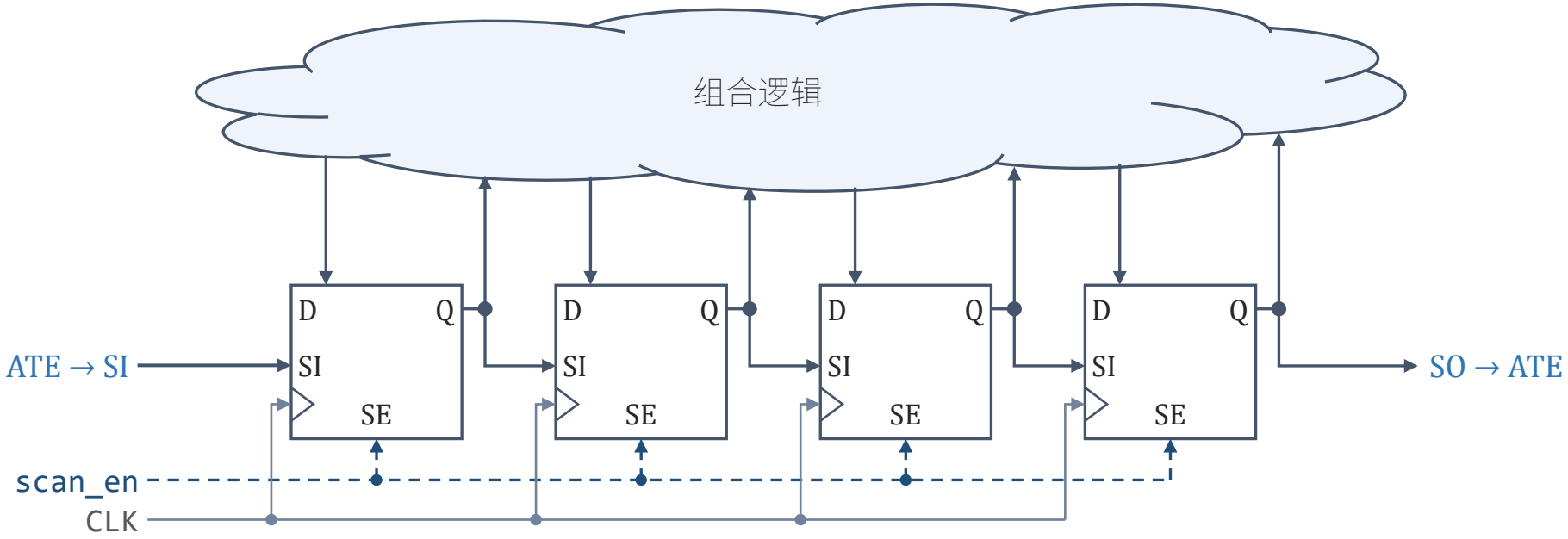


MUX：Multiplexer；DFF：D-Type Flip-Flop；SE：Scan Enable；SI：Scan In；LSSD：Level-Sensitive Scan Design

→ 扫描单元串成链后，一次测试怎样进行？

一条扫描链：**shift in → capture → shift out**

数据从链头 SI（Scan In）移入，经各单元传到链尾 SO（Scan Out）移出。



① **shift in**　　scan_en = 1，已知值 1010 逐拍移入每个 FF

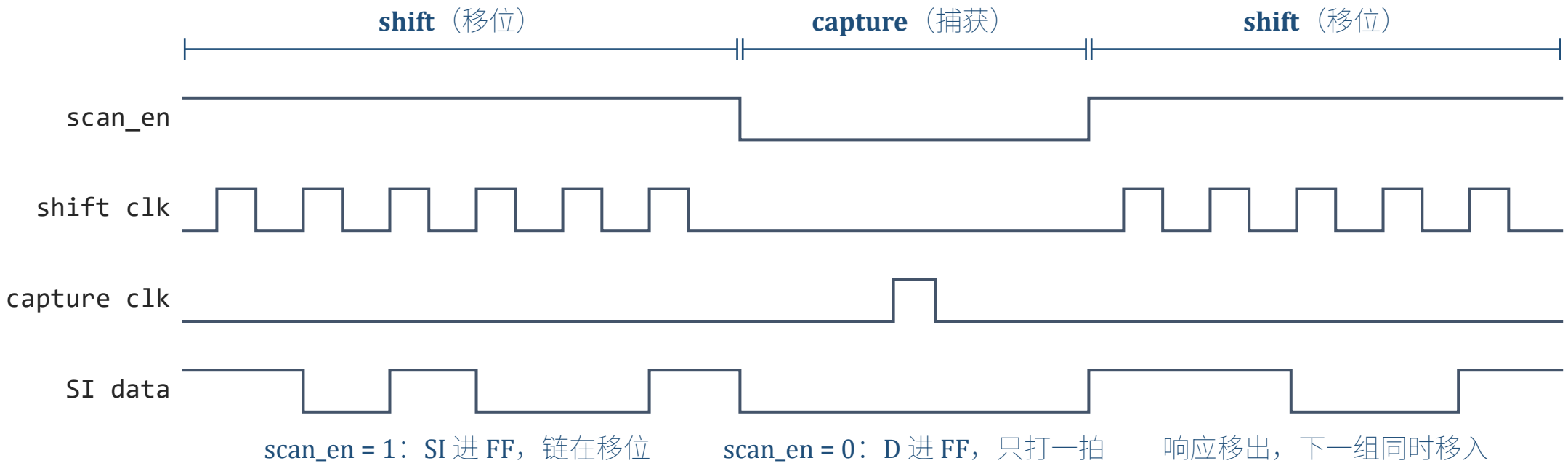
② **capture**　　scan_en = 0，打一拍功能钟，组合逻辑的响应装进 FF

③ **shift out**　　scan_en = 1，响应移出到 ATE 比较；同时下一组已知值移入

→ scan_en 在移位与捕获时各取什么值?

scan_en：移位期间为 1，捕获时通常关闭

scan_en 是动态信号：它随移位与捕获切换，不是模式常量。



load_unload：整个 load_unload 期间 scan_en 须保持为 1，否则报 D1 违例
（除非 async_set_reset_static_disable 已置 1）

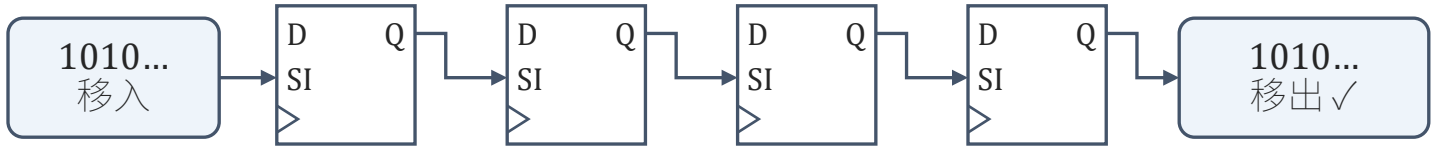
capture：使用 OCC 时 scan_en 在捕获周期被约束为 0；它是动态信号，不由 TDR 驱动

→ 怎样先证明链本身是通的?

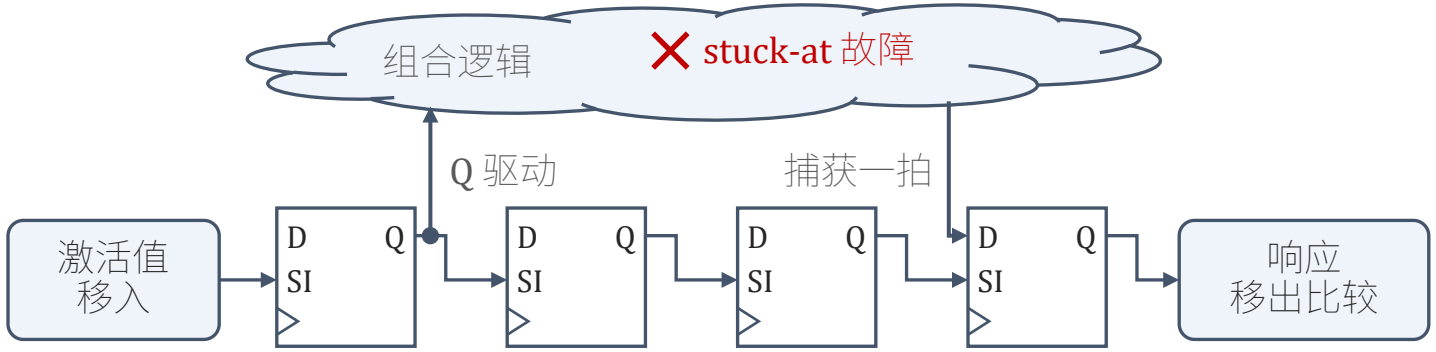
先用 **chain pattern** 证明链通，再谈故障覆盖

chain pattern 只检查移位通路；scan pattern 针对故障模型，含一次捕获。

chain pattern：只测移位连通
移入 1010…，不经捕获直接移出



scan pattern：针对故障模型
移入激活值，捕获一拍后移出

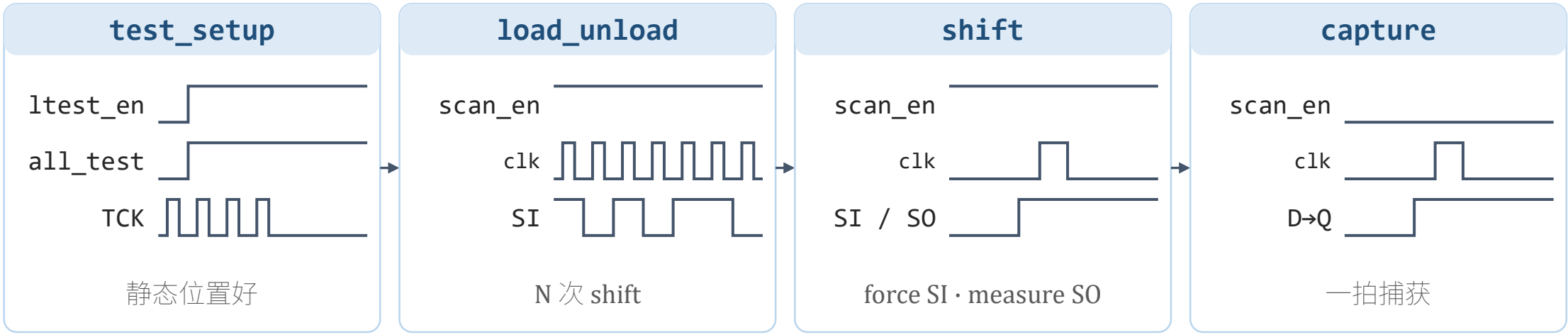


链断了，覆盖率没有意义：先用 chain pattern 证明链通；两类 pattern 分别写出、分别仿真。

→ ATE 与 ATPG 按哪几个测试过程工作?

四个测试过程：test_setup、load_unload、shift、capture

ATE 与 ATPG 都按这四个命名过程执行 pattern。



开头执行一次：
置好非扫描单元与静态位

装载 / 卸载整条链，
内含若干次 shift

每拍打入（force）
并测量（measure）

打一拍捕获钟，
逻辑响应装进 FF

执行顺序：test_setup 一次 → 每个 pattern load_unload（含 N 次 shift）→ capture；
下一次 load_unload 同时卸出上一次的响应

→ 相邻两个 FF 时钟不同时，移位会出什么问题？

跨时钟域移位要插 **lockup**

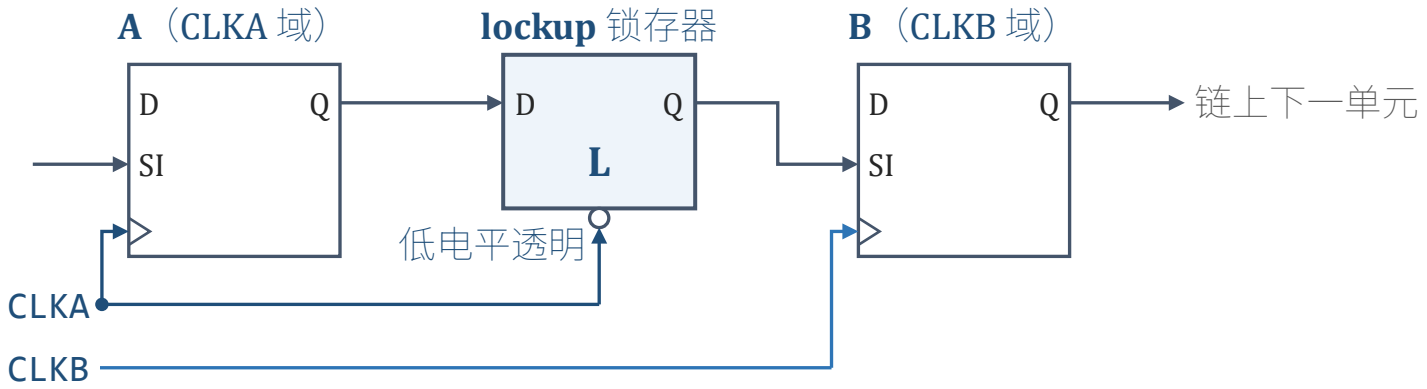
相邻扫描单元时钟不同（或沿不同）时，后一级可能提前采到新值，丢失一位。

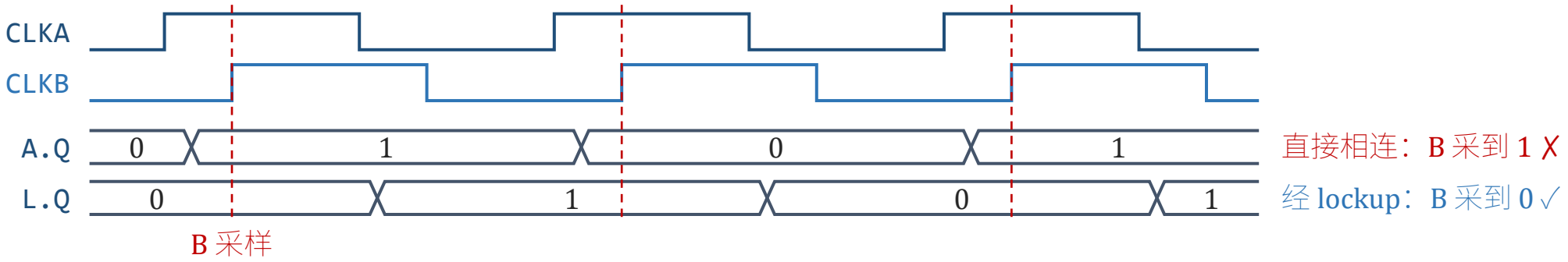
位置：跨时钟域的相邻单元间

器件：默认锁存器，可选触发器

作用：数据晚半拍，B 采到旧值

插入：插链与 EDT 生成时自动加





CLKB 晚到时，A 的新值抢在 B 采样前到达；L 在 CLKA 为低时才放行，数据晚半拍，B 仍采到上一拍的值。

→ TAP：怎样用少量专用引脚进入芯片内部？

JTAG / TAP：少量专用引脚提供标准测试入口

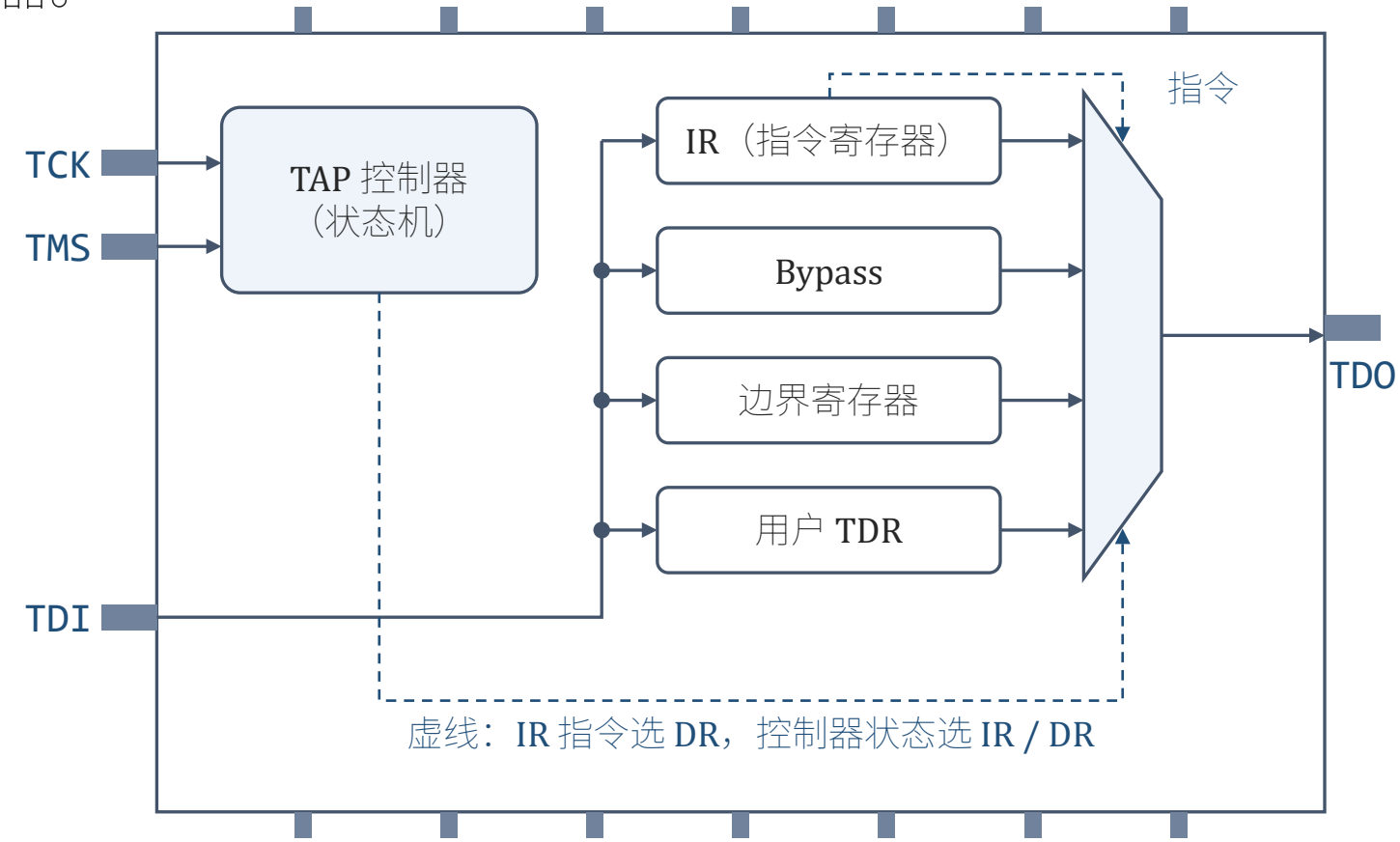
用几根专用引脚接入片内测试寄存器。

TCK / TMS：推动 TAP 状态机

TDI → TDO：串行进出所选寄存器

选通：同一时刻只接一组

标准：IEEE 1149.1

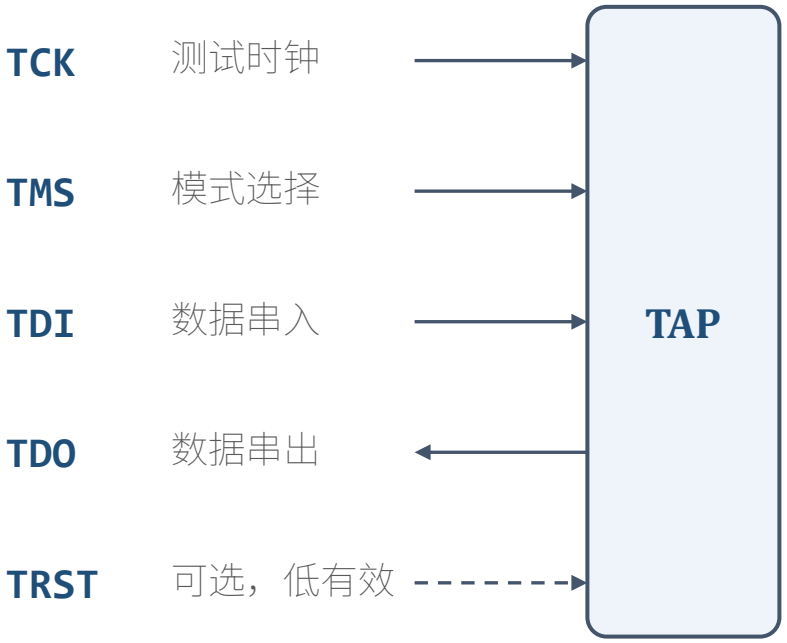


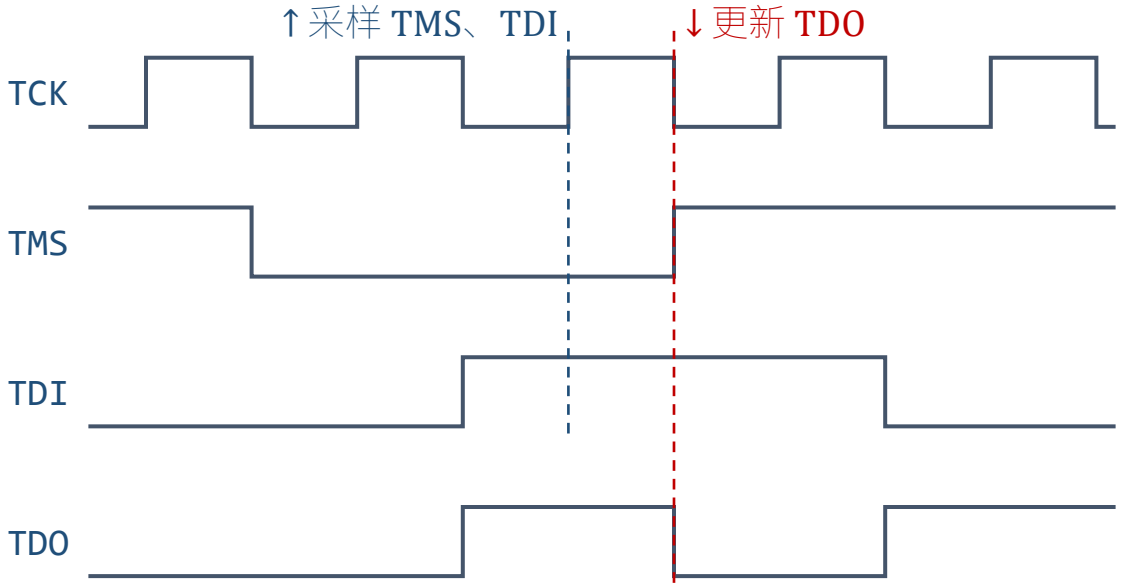
TAP：Test Access Port；JTAG：Joint Test Action Group，IEEE 1149.1 的通称

→ TAP 的四根线各做什么？

TAP 四线：TCK、TMS、TDI、TDO，TRST 可选

四个测试信号由 IEEE 1149.1 定义；TRST 可选，为低有效的异步复位。





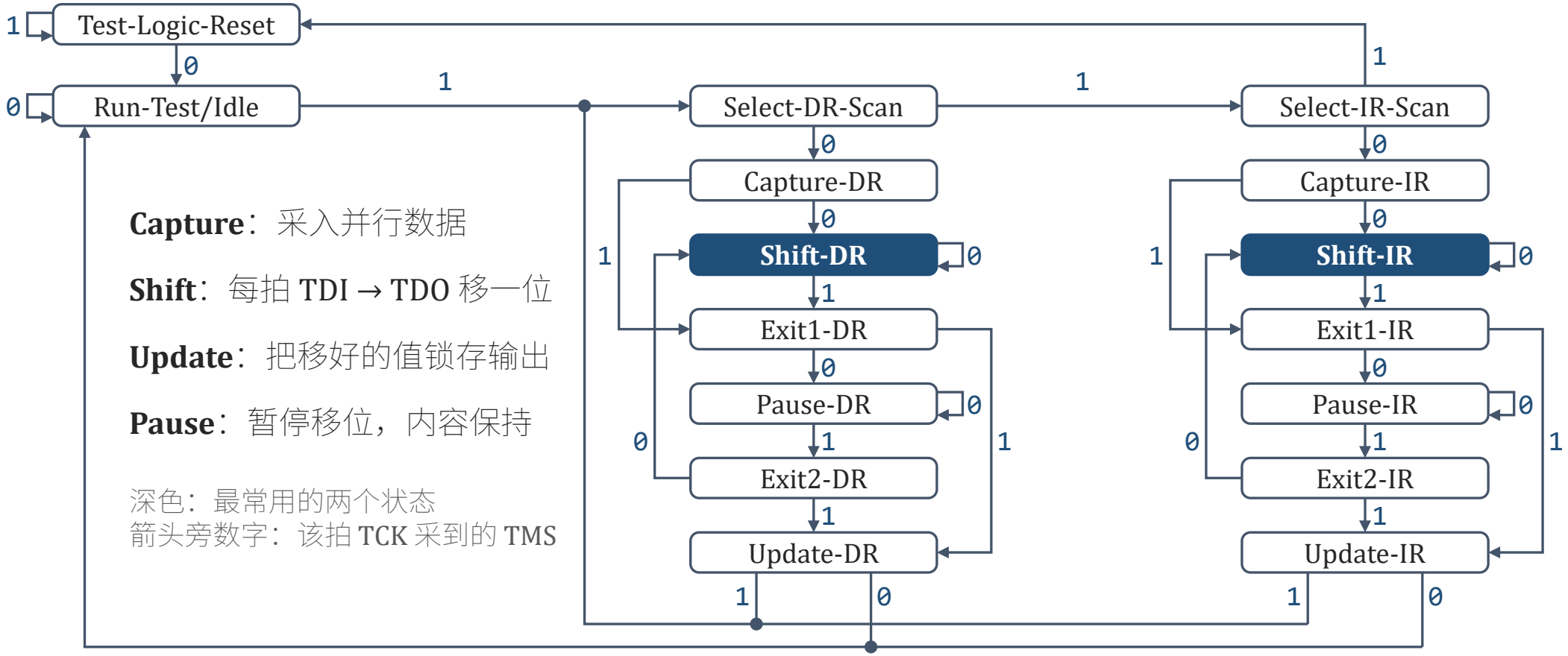
TCK 上升沿采样 TMS 与 TDI，下降沿更新 TDO

TCK：Test Clock；TMS：Test Mode Select；TDI：Test Data In；TDO：Test Data Out；TRST：Test Reset

→ TMS 怎样驱动 TAP 控制器的 16 个状态?

TAP 控制器 16 态：每拍 TCK 由 TMS 决定下一状态

两条主干：IR 扫描、DR 扫描；任意状态下 TMS 连续 5 拍为 1，都回到 Test-Logic-Reset。



→ IR 从哪些 DR 中选通一条？

IR 决定 TDI 与 TDO 之间接哪一条 DR

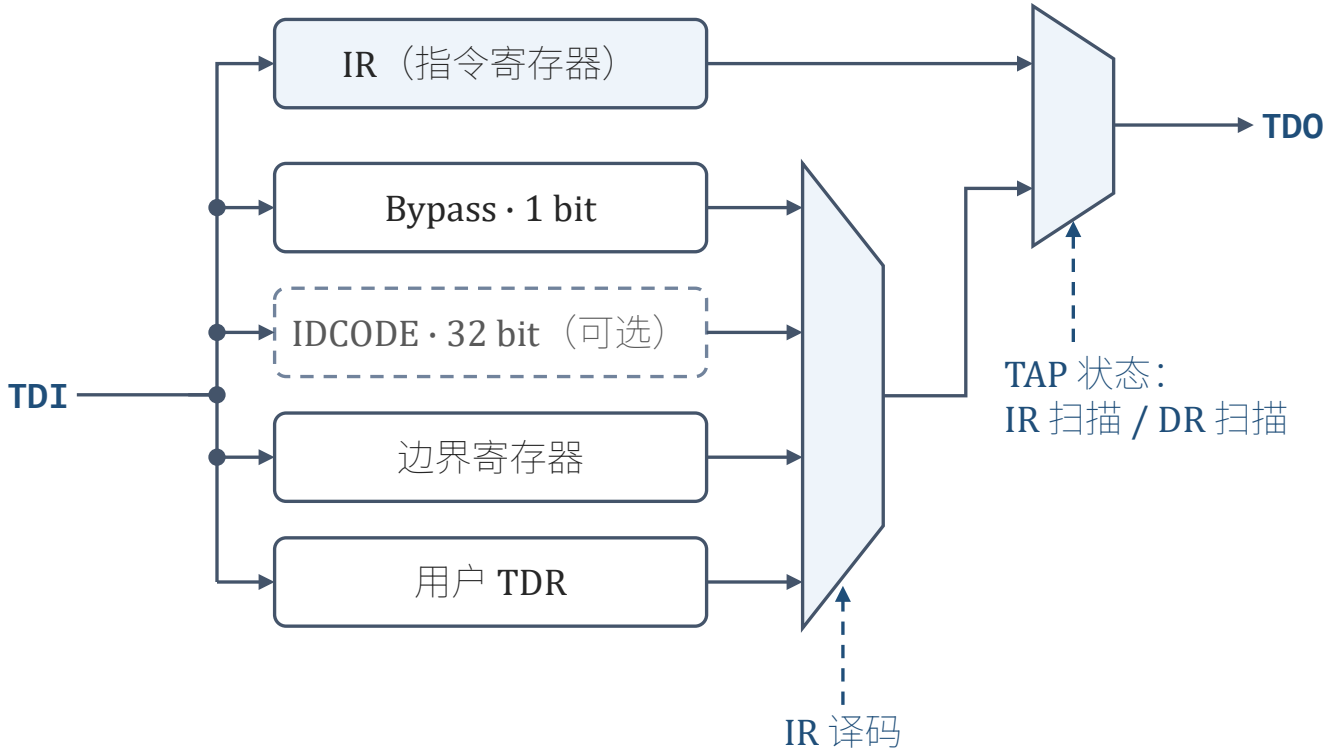
IR 里的指令选出一条 DR 接在 TDI 与 TDO 之间；IR 扫描时则接 IR 本身。

Bypass：只留 1 位，缩短板级链

IDCODE：可选的器件标识寄存器

DR：边界寄存器与用户 TDR 也是 DR

选通：任意时刻只接一条寄存器



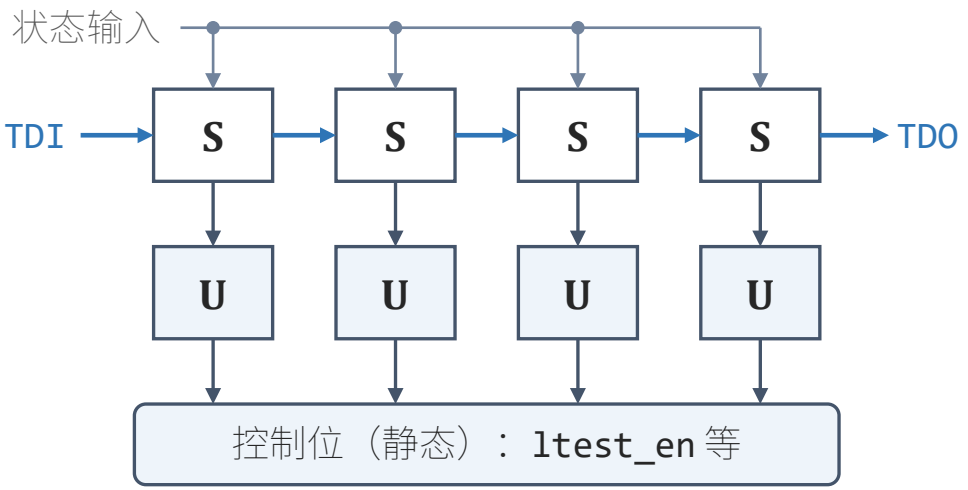
IR：Instruction Register；DR：Data Register

→ TDR 怎样把串行 access 变成并行控制?

TDR：把串行 access 变成 instrument 的并行控制与观测

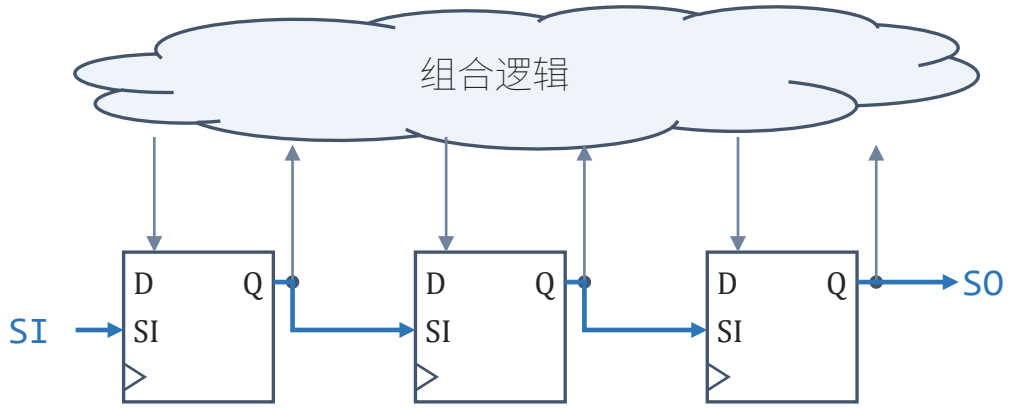
TDR 经 TAP 串行写入后，并行驱动 instrument 的控制位；也能并行采样状态再移出。

TDR：移位级 + 更新级



Shift-DR 逐位移入 S 级，Update-DR 一次更新 U 级
移位期间 U 级输出保持不变

逻辑扫描链：单级、无更新级



功能 FF 兼作扫描单元，移位时 Q 随之翻转
只服务 ATPG：装入激活值、收回响应

TDR：Test Data Register；S：移位级（shift）；U：更新级（update）

→ 边界扫描怎样把芯片周边串成一条寄存器？

边界扫描：沿芯片周边串成一条边界寄存器

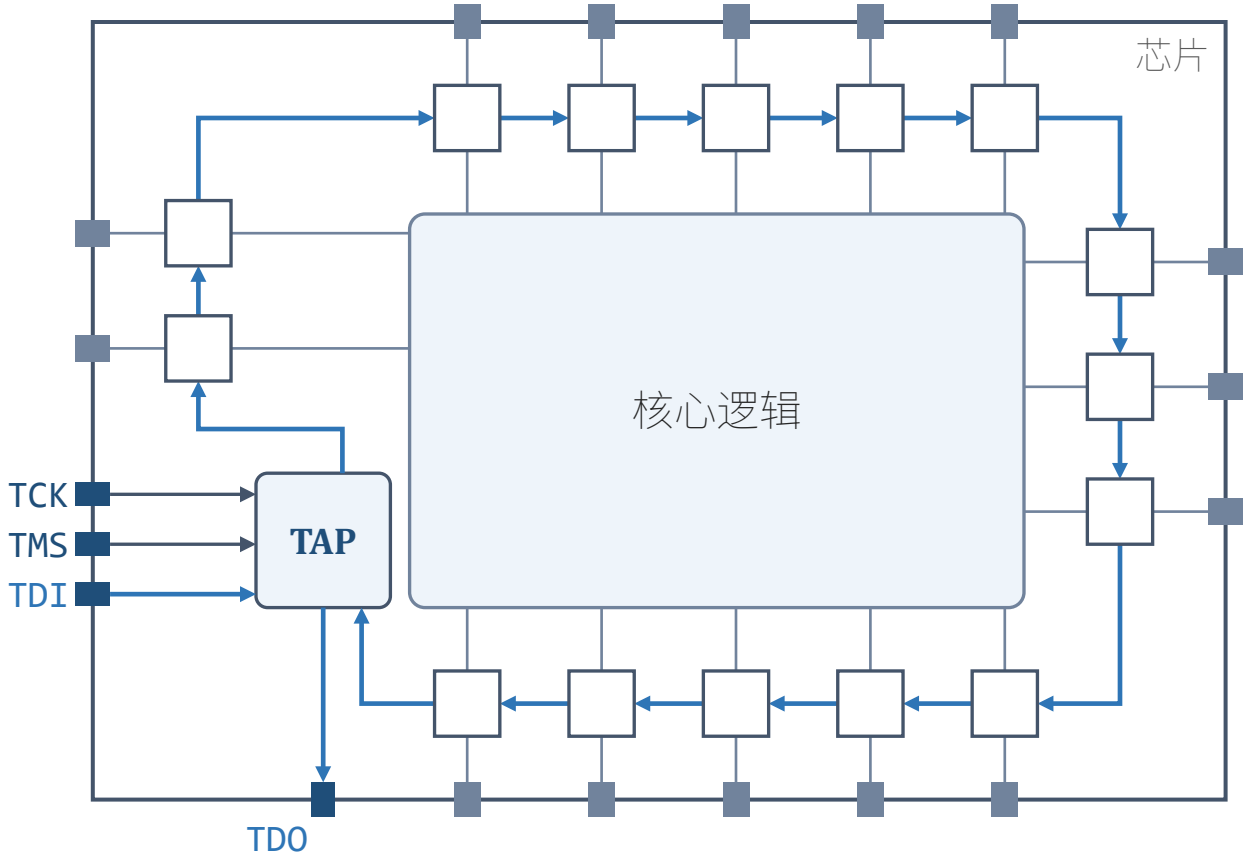
在核心与每个 I/O 之间插入一个边界单元（boundary cell）。

装卸：可串行，也可并行装卸 I/O 数据

分工：核心另有内部扫描链，互不串接

归属：整条边界寄存器是 TAP 下的一个 DR

蓝线：边界寄存器由 TAP 接入，
沿芯片绕一圈，再经 TAP 出 TDO



→ 边界扫描的两条强制指令各做什么？

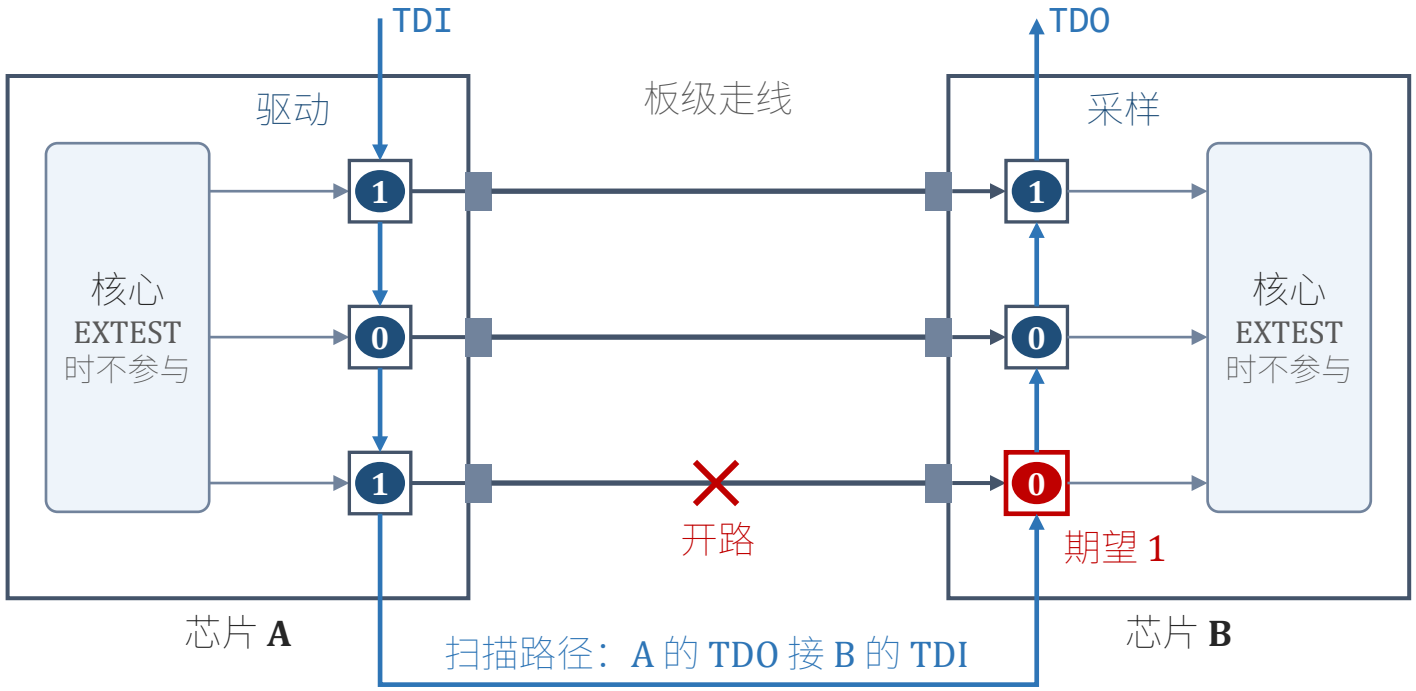
EXTEST 测板级互连，SAMPLE/PRELOAD 采样与预装

两者都是 IEEE 1149.1 的强制指令。

EXTEST：测器件外面的板级互连，是边界扫描的主测试指令

SAMPLE / PRELOAD：正常工作时把 I/O 采进边界寄存器

1149.1-2001 起：拆成两条指令，装载时工具优先用 PRELOAD

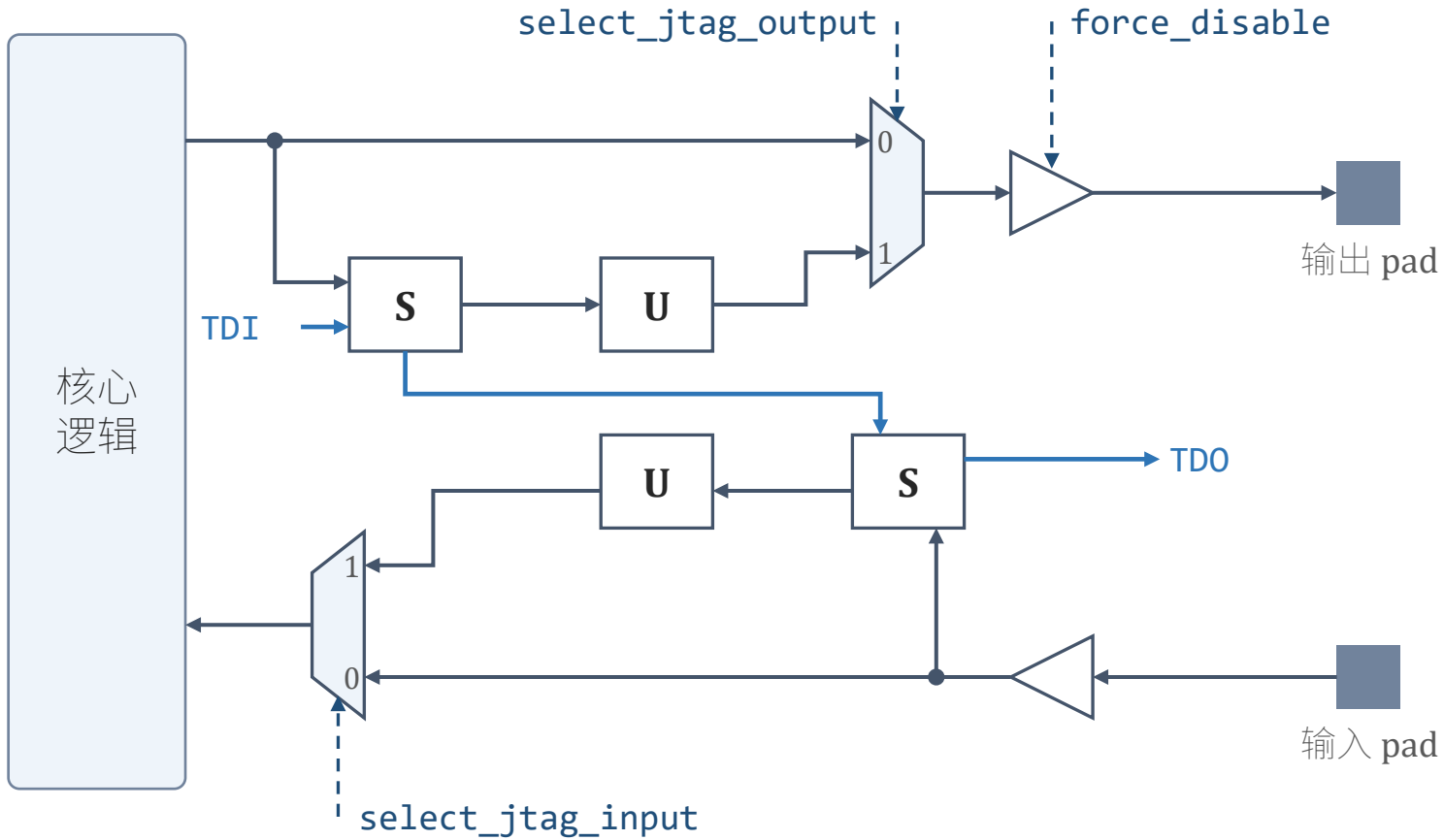


① **PRELOAD**：驱动值装入 A → ② **EXTEST**：A 驱动走线，B 采样 → ③ 移出：采样值与期望比较

→ 同一套边界单元还有哪种用法?

边界单元两种用法：对外测连线，对内隔离 **pad**

同一个边界单元：**EXTEST** 时对外驱动与采样，逻辑测试时对内钳住或隔离 pad。



S 与 **U** 两级

S：Capture-DR 采样，Shift-DR 移位

U：Update-DR 才更新，移位时不变

对外：**EXTEST**

`select_jtag_output` = 1：

pad 由 U 驱动，输入单元 S 采样 pad

对内：输出

`output_pad_disable` = 1：pad 高阻

`bscan_clamp_enable` = 1：钳在 U 的值

对内：输入

`bscan_input_isolation_enable`

置 1：核心输入改取边界单元 U 的值

→ 板级工具怎样知道器件的边界扫描结构?

BSDL 描述器件的指令、边界寄存器与管脚映射

BSDL 是描述器件边界扫描结构的文本，板级工具与 ATE 据此操作这颗器件。

文件内容：IR 长度与指令码、
边界寄存器长度与单元类型、
管脚与边界单元的对应关系

文件来源：边界扫描插入后，
由 Tessent 写出，
符合 IEEE 1149.1-2001



没有 BSDL，板级工具就不知道用哪个指令码把这颗器件置于 BYPASS。

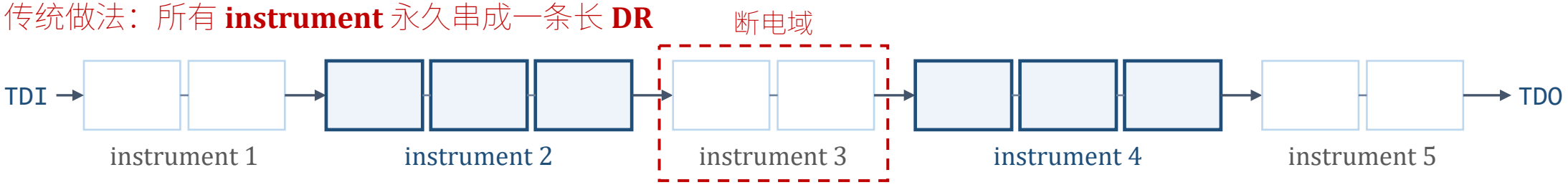
BSDL：Boundary Scan Description Language，由 IEEE 1149.1 定义；BSR：Boundary Scan Register（边界寄存器）

→ 片上 instrument 很多时，怎样避免扫描路径过长？

IJTAG（IEEE 1687）只接入当前用到的 instrument 段

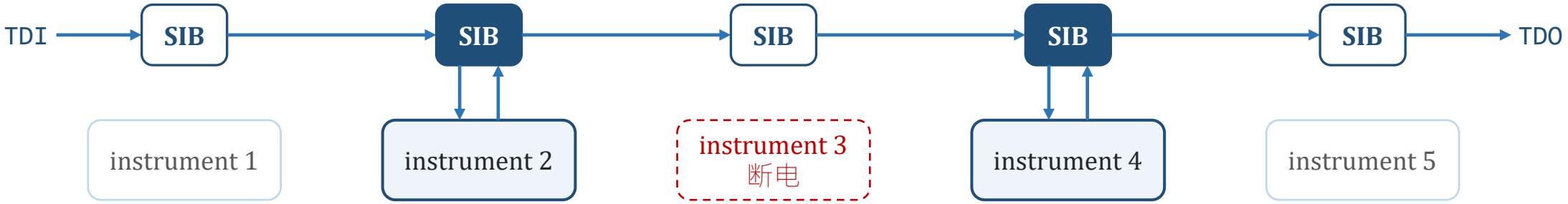
片上 instrument 多时，全部串成一条 DR 既长又易断；IEEE 1687 改为按需接入。

传统做法：所有 **instrument** 永久串成一条长 **DR**



只用 instrument 2、4 也要移过全部 12 位；instrument 3 断电时，整条链断开

1687：带开关的短路径，只打开用到的两个 **instrument**



关闭的 SIB 只占 1 位，打开的 SIB 才把 instrument 段接入；断电 instrument 的 SIB 保持关闭，链路不断。

IJTAG：Internal Joint Test Action Group（IEEE 1687）

→ access 网络的连接与操作怎样描述?

ICL 描述连接，PDL 描述操作

Tessent 读入 ICL 得到 access 网络的结构，再把 PDL 中的读写步骤转换成 TAP 上的扫描序列。

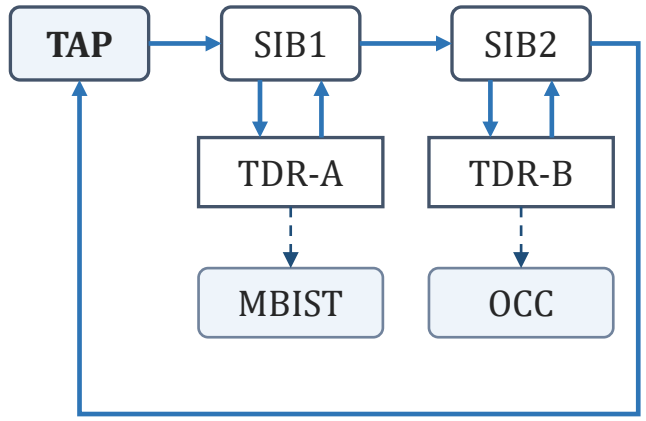
ICL：描述 instrument 的接口，以及 TAP、SIB、TDR 间的连接

PDL：在 instrument 端口上写 `iWrite`、`iRead`、`iApply`

重定向：工具沿 ICL 把 PDL 转换成 TAP 上的扫描操作

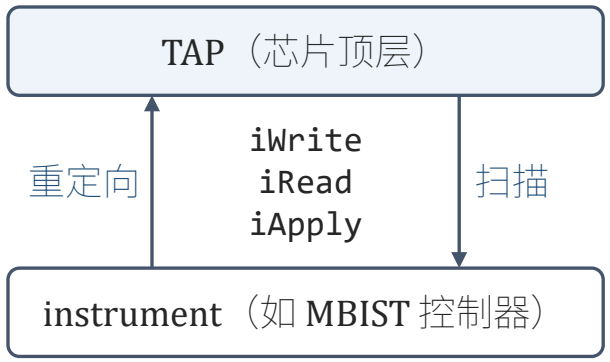
标准：Tessent IJTAG
遵循 IEEE 1687-2014

ICL：**access** 网络的结构



蓝线：扫描路径；虚线：TDR 数据端口

PDL：**instrument** 的操作



在 instrument 端口上写 PDL，
工具沿 ICL 重定向到 TAP

PDL 只需在 instrument 层写一次；接入更大的网络后，工具沿 ICL 自动重定向，上层无需重写。

ICL：Instrument Connectivity Language；PDL：Procedural Description Language

→ SIB 怎样把子网接入或移出扫描路径？

SIB：一位控制位决定子网是否在扫描路径上

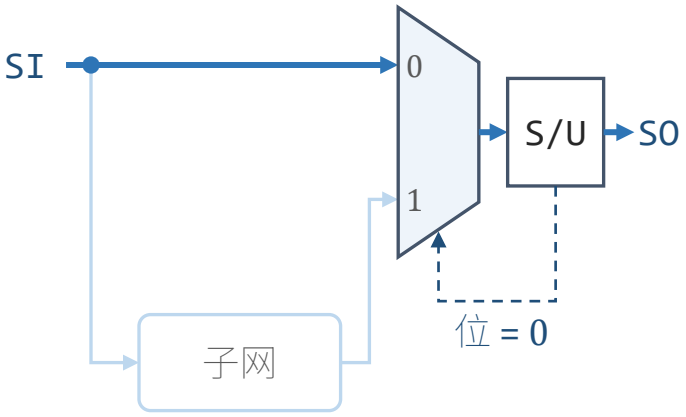
SIB 让扫描路径的长度随需改变：只打开要 access 的 instrument 所在的段。

关闭：SI → SO 直通，
路径只增加 1 位

打开：路径先经过子网，
再回到 SIB

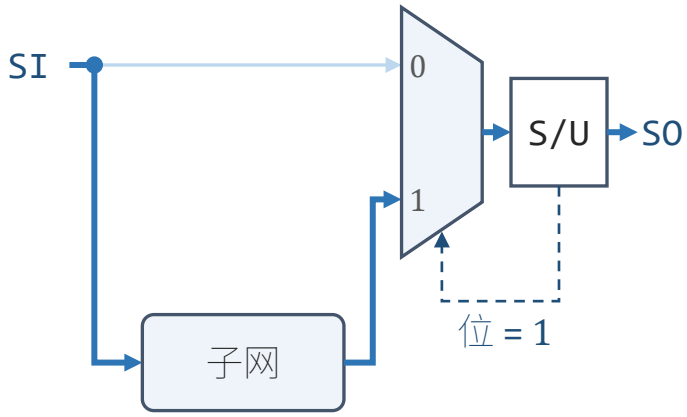
结构：一个移位-更新位
加一个二选一 MUX

关闭：直通



SI → SO：1 位

打开：接入子网



SI → SO：1 位 + 子网的位数

SIB 控制位先移入，在 Update-DR 时生效；从下一次扫描起，路径随之改变。

SIB：Segment Insertion Bit

→ 除了 SIB，路径长度还能怎样改变？

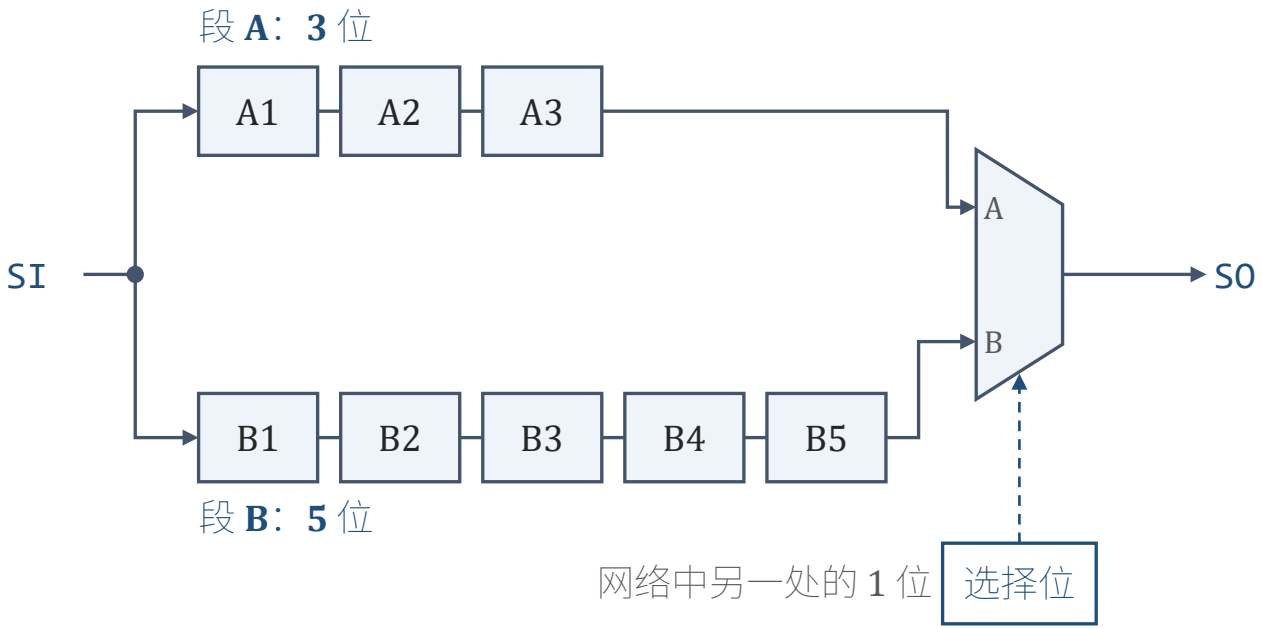
ScanMux 选择扫描段，与 SIB 一起使路径长度可变

SIB 决定一段接不接入；ScanMux 在几段扫描路径中选一段接入。

选段：选 A 或 B，路径长度随之改变

选择方式：由工具在重定向时决定，
并避开会断开扫描路径的取值

选择位：也是 access 网络中的一位



选 A 时 SI → SO 为 3 位，选 B 时为 5 位：同一个物理网络，路径长度可变。

→ 各类 instrument 怎样接入 access 网络?

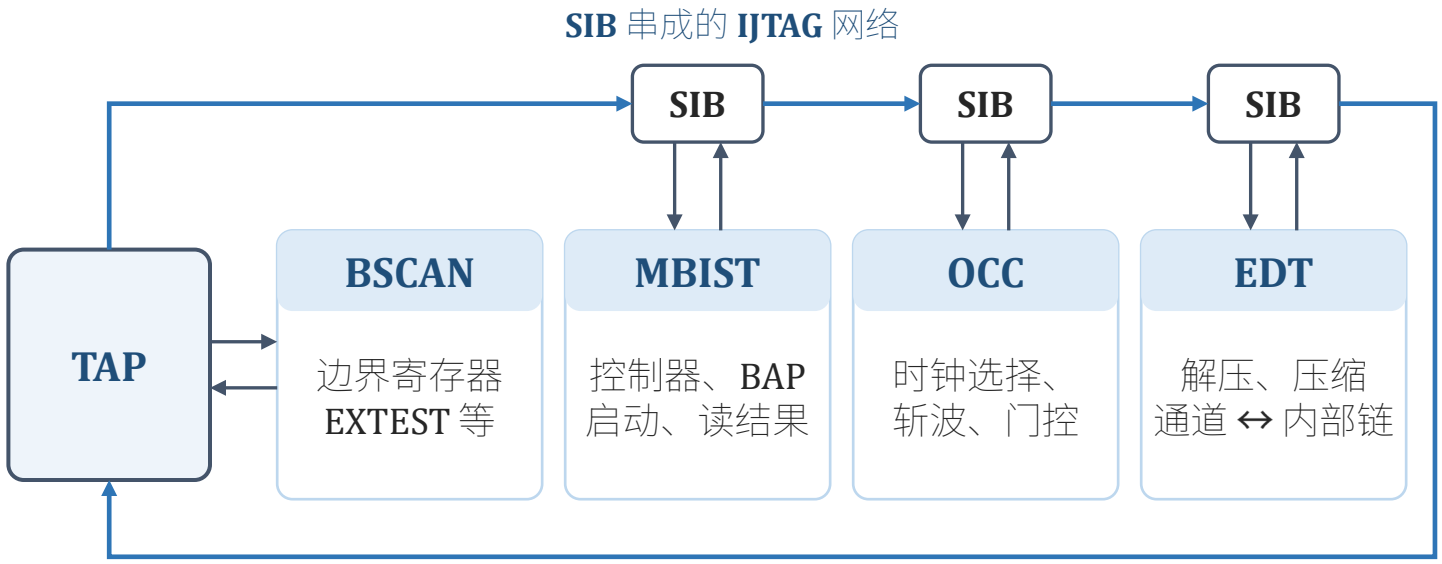
先建 access 网络，再经 SIB 接入 MBIST、OCC、EDT

TAP 提供对外通道；每个 SIB 决定其下的 instrument 此刻是否在扫描路径上。

TAP：芯片对外的测试端口，含状态机与指令寄存器

BSCAN：经 HostBscan 接口直接接 TAP，不经 SIB

SIB：MBIST、OCC、EDT 各经一个 SIB 接入



SIB 由 TAP 经 IJTAG 网络写入；打开后，其下 instrument 的 TDR 才进入扫描路径。

BSCAN：Boundary Scan；BAP：BIST Access Port；HostBscan：TAP 上连接边界扫描链的接口

→ 存储器为什么要在片上自测?

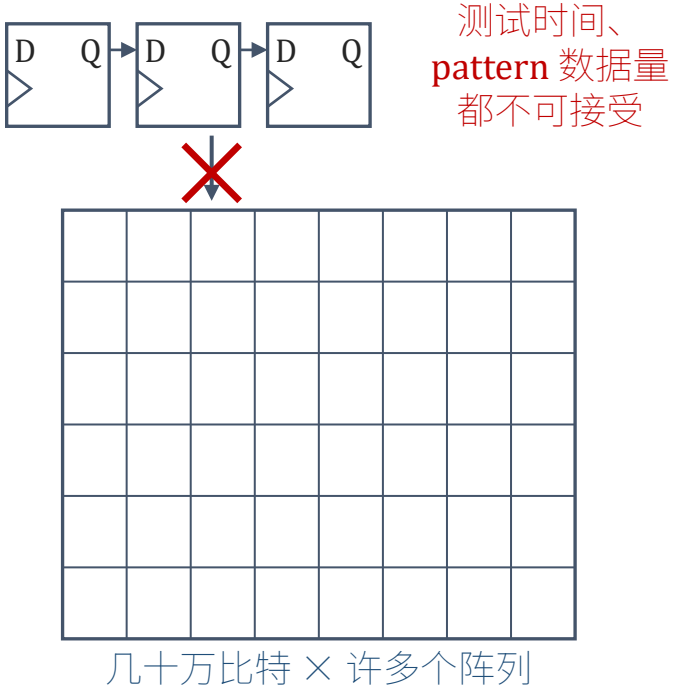
MBIST：存储器在片上自测，外部只启动与读结果

片上：算法生成地址、数据与读写

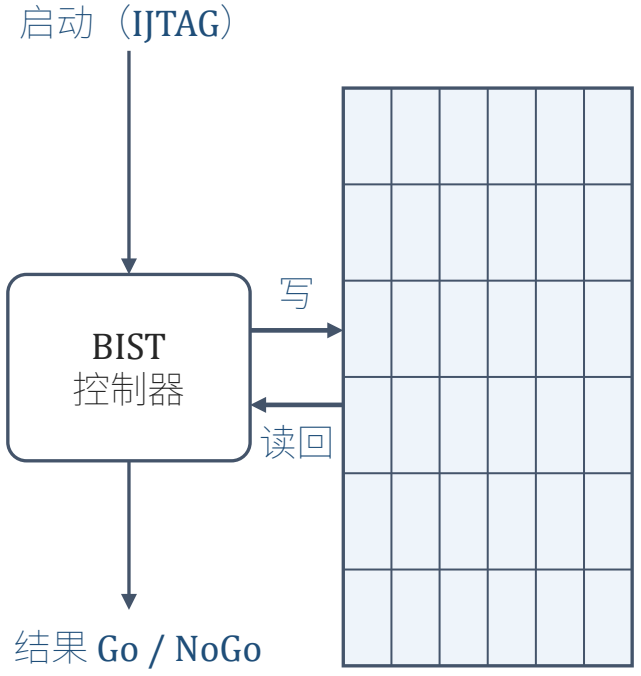
外部：只启动测试、读取结果

原因：比特数太多，扫描写入太慢

若用扫描逐地址写入



MBIST：控制器就在阵列旁边



MBIST：Memory Built-In Self-Test

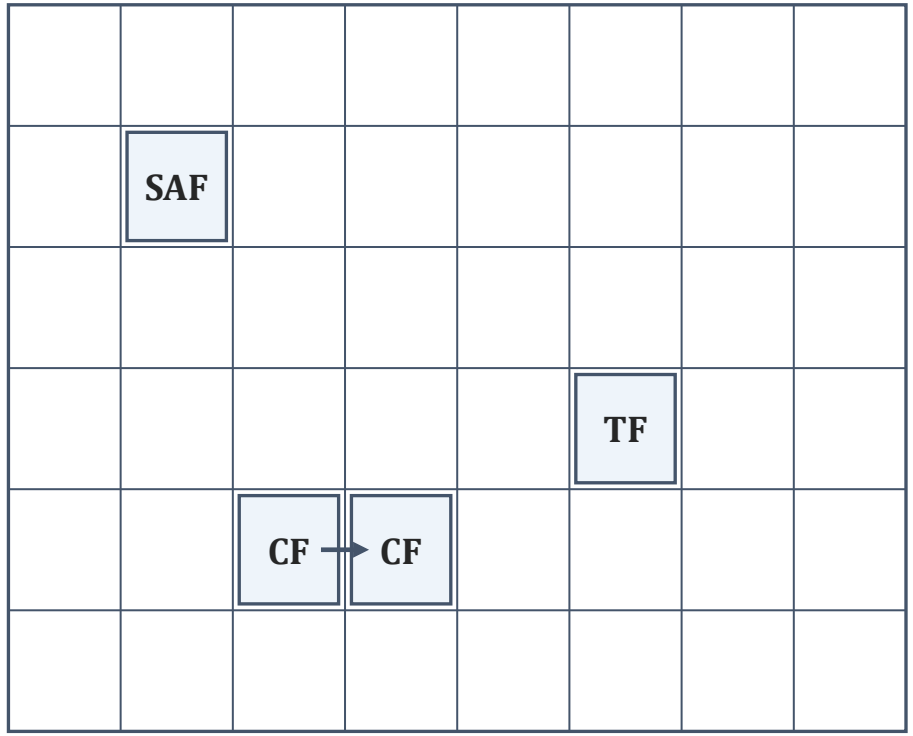
→ 存储器的故障与逻辑故障有何不同?

存储器故障不同于逻辑故障，**stuck-at** 模型不够

逻辑 ATPG 通过，不能证明存储器无故障

存储器需要独立的算法与测试通路

SAF	单元恒为 0 或恒为 1
TF	单元无法完成 0→1 或 1→0 翻转
CF	写一个单元会改变另一个单元
AF / 保持	地址译码出错；数据无法保持



March 类算法就是为这些机制设计的读写序列；stuck-at 只覆盖其中一小部分。

SAF：stuck-at fault；TF：transition fault；CF：coupling fault；AF：address-decoder fault

→ March 算法怎样覆盖这些故障?

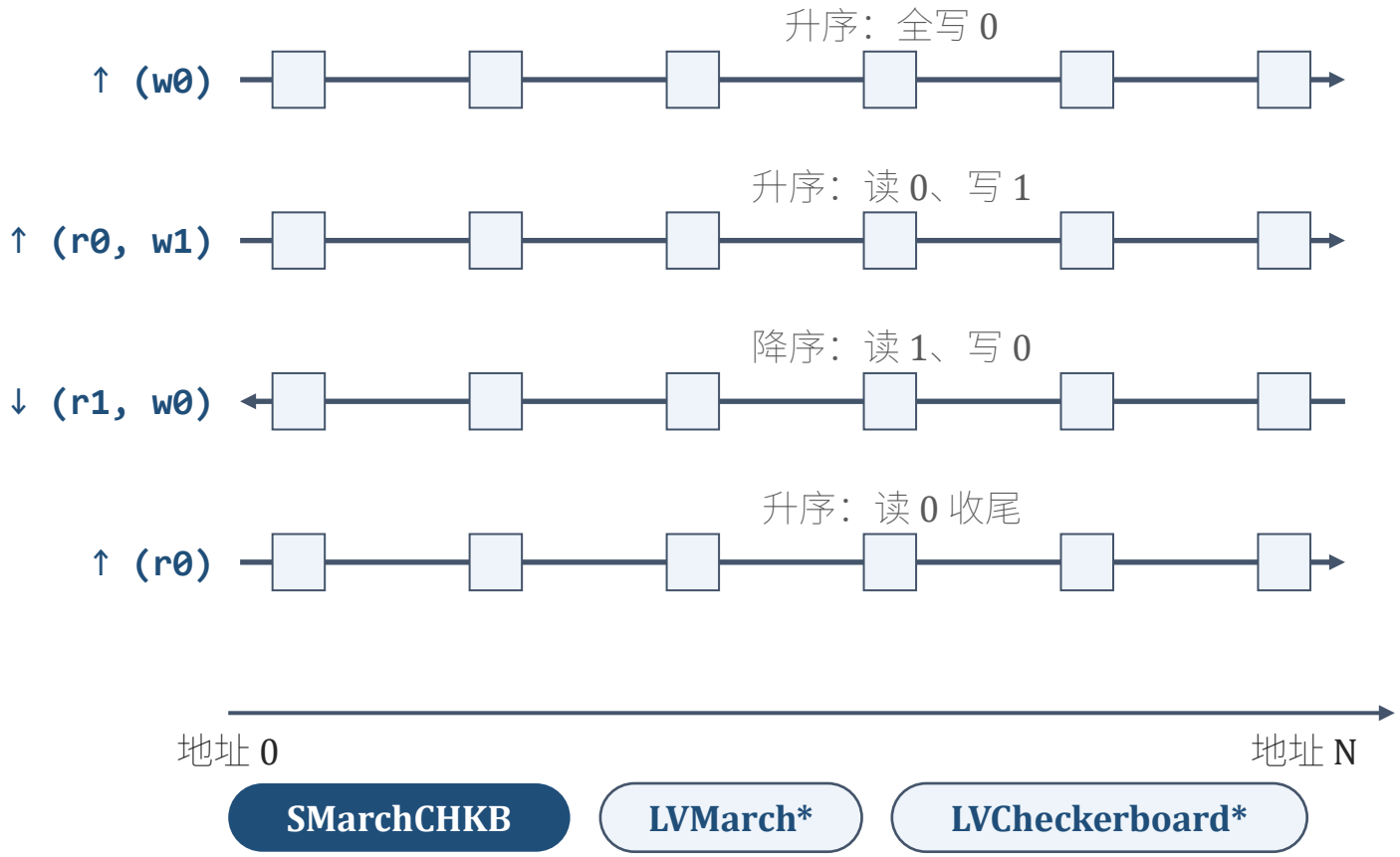
March 算法：每个元素按一个地址方向做一串读写

元素：一个地址方向 + 一串读写操作

判定：读出值须等于上一元素写入的值

算法库：常用 SMarchCHKB 及其 ci / cil / vcd 变体

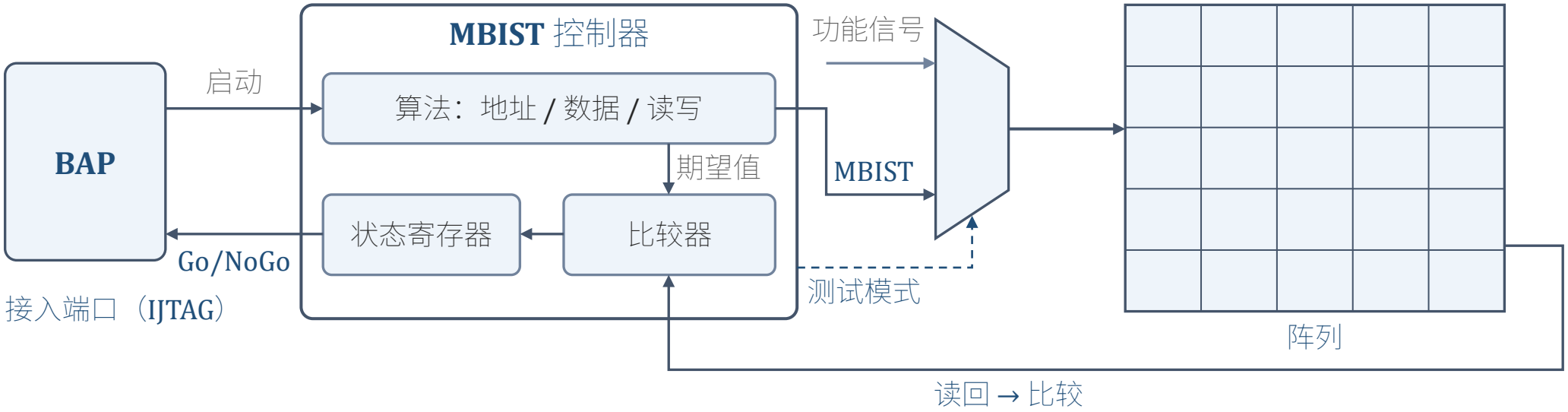
一条 **March** 序列示意：元素按顺序执行，每个元素扫过全部地址



→ MBIST 控制器怎样接入并读出结果？

控制器执行算法，**BAP** 提供 **access**，接口多路切换信号

BAP 经 IJTAG 启动 MBIST 控制器；控制器读写阵列、比较结果，状态经 BAP 读出。



Go/NoGo：比较器的失败累积到状态寄存器，经 BAP 读出

接口多路：功能路径与 MBIST 路径二选一

可选能力：诊断与 BISR

BAP：BIST Access Port；BISR：Built-In Self-Repair

→ 逻辑测试时，存储器阵列怎样处理？

memory_bypass_en：逻辑测试默认旁路阵列，阵列交给 MBIST

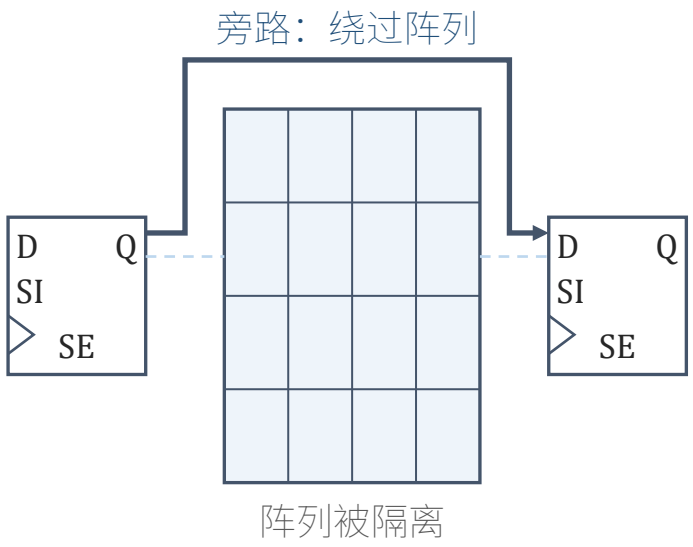
阵列输出在逻辑测试中难以控制、容易带 X，默认把阵列旁路。

取 **1**（默认）：旁路阵列，
下游逻辑不再受阵列 X 影响

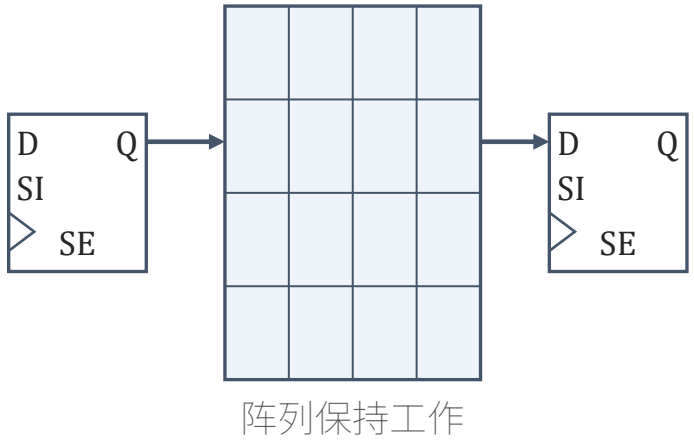
取 **0**：阵列保持工作，
用 multi-load pattern 测端口

分工：阵列本身由 MBIST 测，
逻辑 ATPG 只覆盖到端口

逻辑测试：旁路阵列



逻辑测试：阵列保持工作



`memory_bypass_en = 1`

`memory_bypass_en = 0`

→ DC 测试与 AC 测试有何不同?

DC 测试慢速捕获，AC 测试至速捕获

DC、AC 按捕获速度与故障模型区分：DC 检出静态故障，AC 检出延时故障。

DC 测试（慢速）

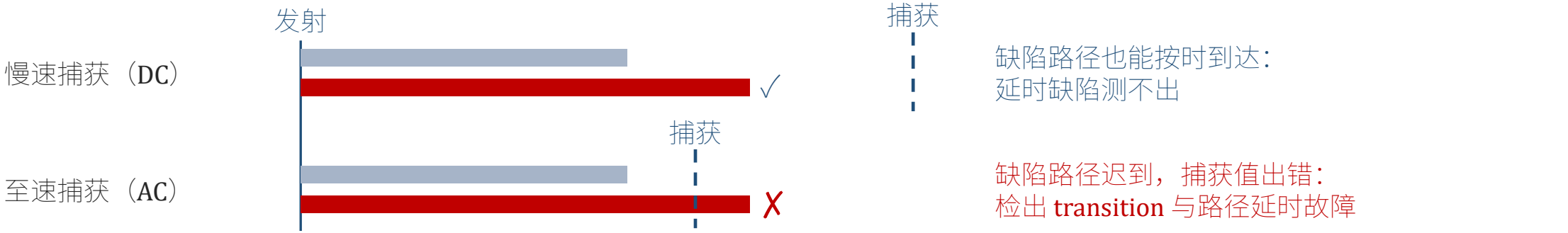
捕获：慢速时钟，时间余量充足
故障模型：stuck-at 等静态模型
检不出：小延时缺陷

AC 测试（至速）

捕获：两拍至速时钟（由 OCC 产生）
故障模型：transition、path delay
检出：小延时缺陷

同一条路径，两种捕获间隔

灰：正常路径延时　红：带小延时缺陷



DC 与 AC 分别生成 pattern、分别统计覆盖率：故障模型不同，结果不能互相沿用。

→ 至速测试怎样发射 transition?

至速测试：**LOC** 用功能路径发射，**LOS** 用最后一拍移位发射

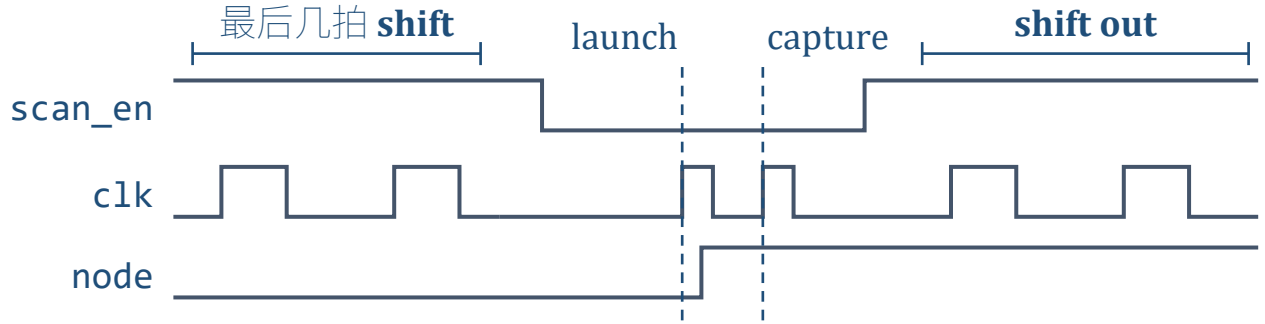
至速测试先发射（launch）一个 transition，下一拍立即捕获（capture）。

LOC：由第一拍捕获钟发射，
`scan_en` 可慢速切换，更常用

LOS：最后一拍移位即发射，
`scan_en` 须在一个功能周期内拉低

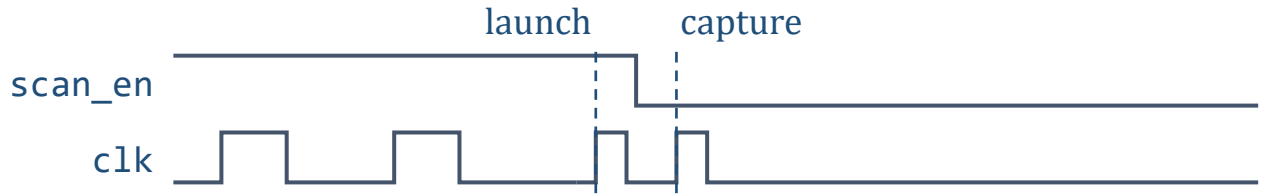
本课：以 LOC 为主；
做 LOS 时用 `se_pipeline_en`
把 `scan_en` 流水化

LOC：**Launch-Off-Capture**



`scan_en` 已为 0：两拍快钟都走功能路径，第一拍即 launch

LOS：**Launch-Off-Shift**



最后一拍 shift 兼作 launch：`scan_en` 须在一个功能周期内拉低

LOC：Launch-Off-Capture；LOS：Launch-Off-Shift

→ 两拍至速时钟在片上怎样产生？

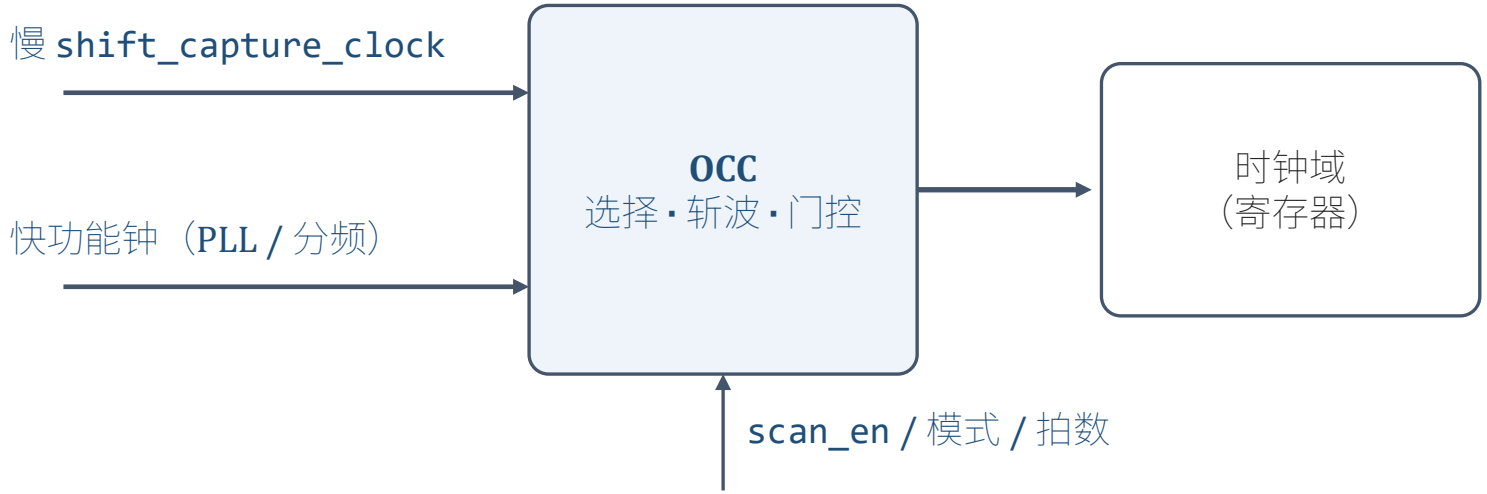
OCC：在片上生成移位慢钟与两拍至速捕获钟

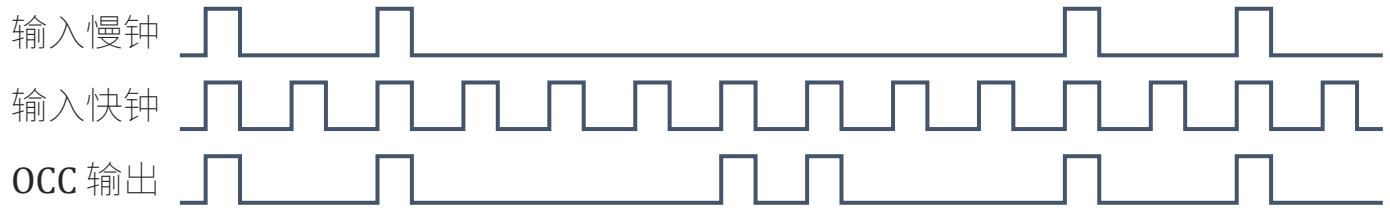
标准 **OCC**：时钟选择、斩波与门控

移位：`shift_capture_clock` 提供慢速移位时钟

捕获：只在捕获窗内放行两拍至速功能时钟

层次化：有了核内 OCC，pattern 可重定向到顶层





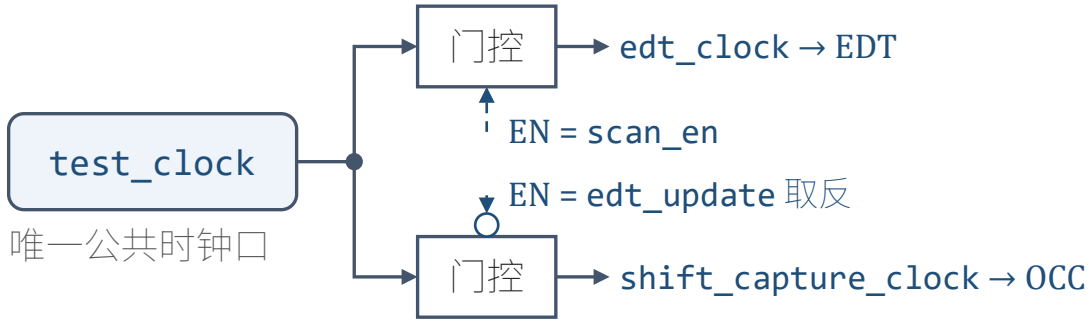
移位用慢钟；捕获窗内只放行两拍完整的快钟脉冲

OCC：On-Chip Clock Controller（clock selection、chopping、gating）；PLL：Phase-Locked Loop（片上时钟源）

→ 测试时钟从哪个端口进来?

推荐只接一个 test_clock 口，另两个时钟由它门控派生

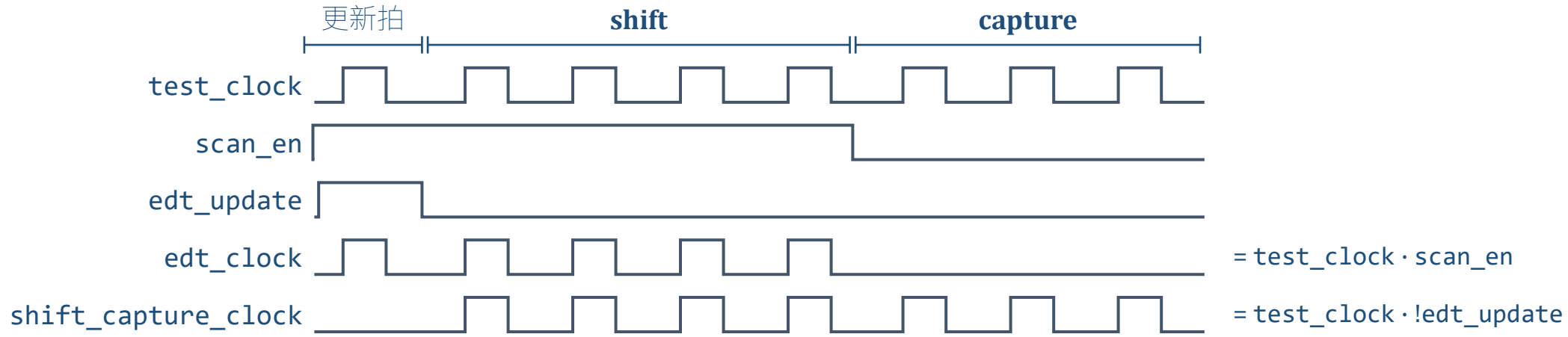
两个派生钟同源于 test_clock，由 scan_en、edt_update 决定哪一拍放行。



更新拍：edt_update = 1，只打 edt_clock

捕获期：scan_en = 0，只打 shift_capture_clock

不复用功能钟：EDT 初始化要额外几拍，
不应扰动功能时钟域



→ 怎样用少量通道驱动大量扫描链?

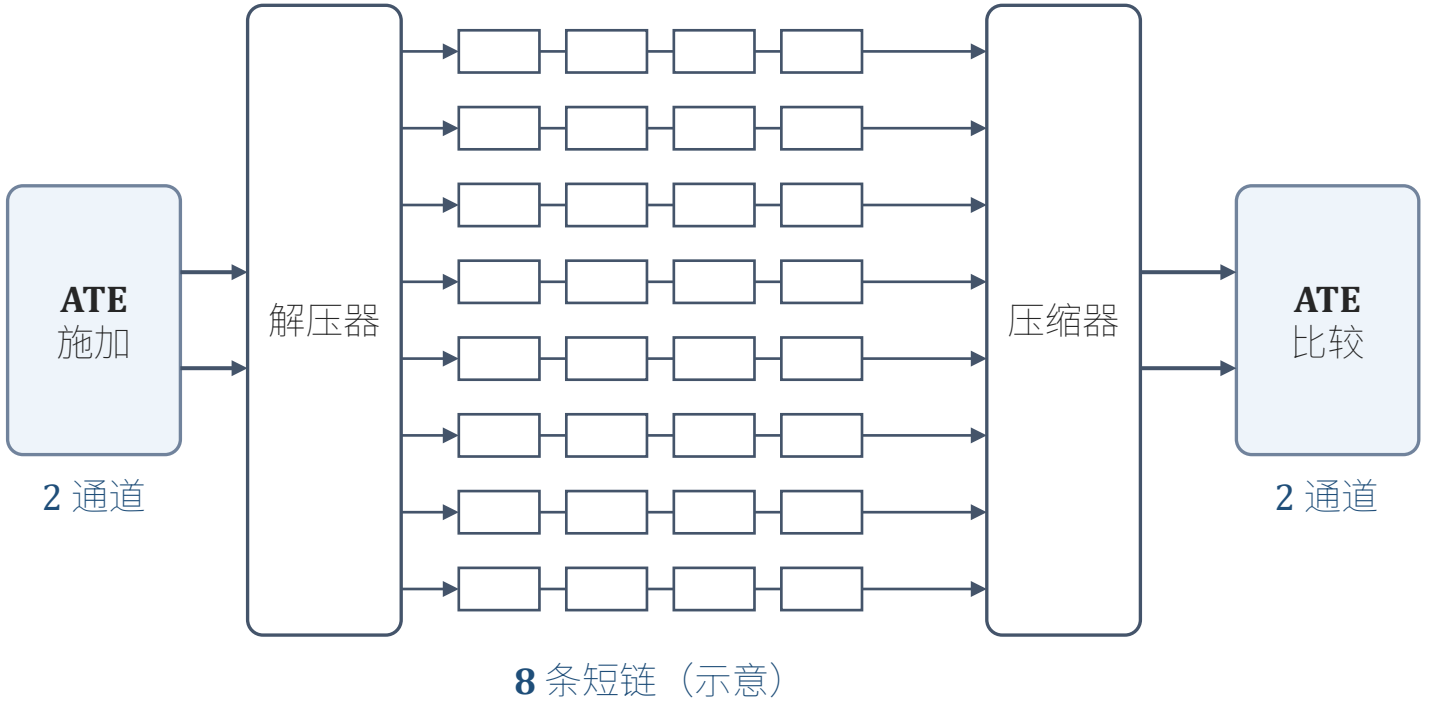
EDT 扫描压缩：少量外部通道驱动多条内部短链

解压器把少数通道的数据展开到多条短链，压缩器再把响应压回少数通道。

通道：外部测试通道保持较少

短链：内部多条短链并行移位

收益：数据量与移位时间都下降



示意：32 个扫描单元。不压缩：2 条 16 位链，每个 pattern 移 16 拍；EDT：8 条 4 位短链，约 4 拍 + 初始化。

EDT：Embedded Deterministic Test

→ 外部通道与内部链的数量关系怎样？

外部通道接 ATE，内部扫描链串接扫描单元

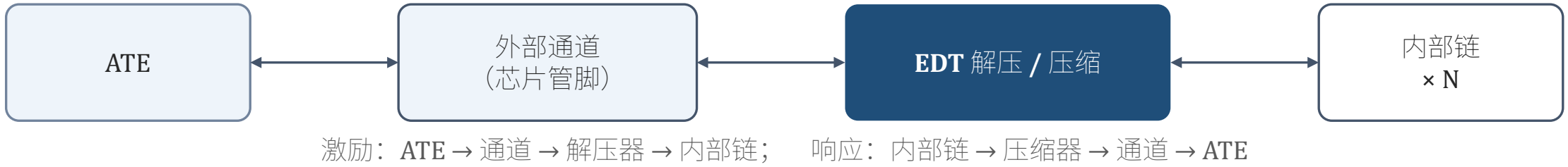
ATE 只接触外部通道；内部链的数目与长度只在芯片内部可见。

参数：通道数、内部链数、
最长链长度

工具：据此生成 EDT 逻辑，
并参考链首、链尾单元的时钟

说明：表中数字仅为示意

	外部通道（芯片管脚）	内部扫描链
输入侧	2 条	N 条（很多）
输出侧	2 条	N 条（很多）
长度	移位拍数 = 最长链长度 + 少量附加拍（初始化等）	最长链 L 位
可见范围	ATE、pattern 文件	仅芯片内部



→ 压缩数据怎样展开到多条内部链?

EDT 解压器：LFSM 与移相器把数据展开到多条内部链

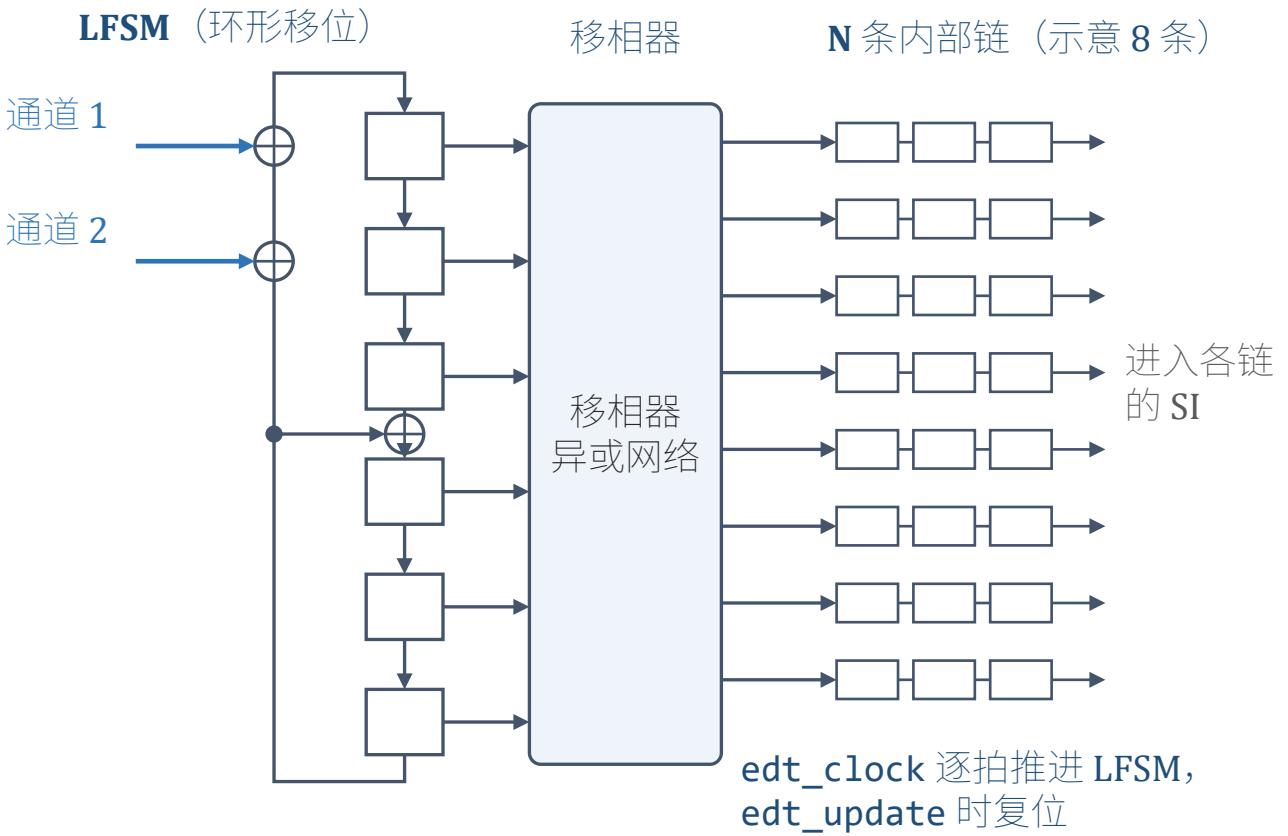
通道数据逐拍注入 LFSM，移相器再把它的状态展开成 N 条链的移入值。

结构：LFSM 加移相器，
合称解压器

注入：每拍把通道数据
异或进 LFSM

展开：移相器组合 LFSM 各位，
送往 N 条内部链

复位：每个 pattern 装载前，
在 `edt_update` 拍复位



LFSM：Linear Finite State Machine（线性有限状态机）

→ 内部链的响应怎样压回少数通道？

EDT 压缩器：多条 SO 压回少数通道，掩码挡住 X

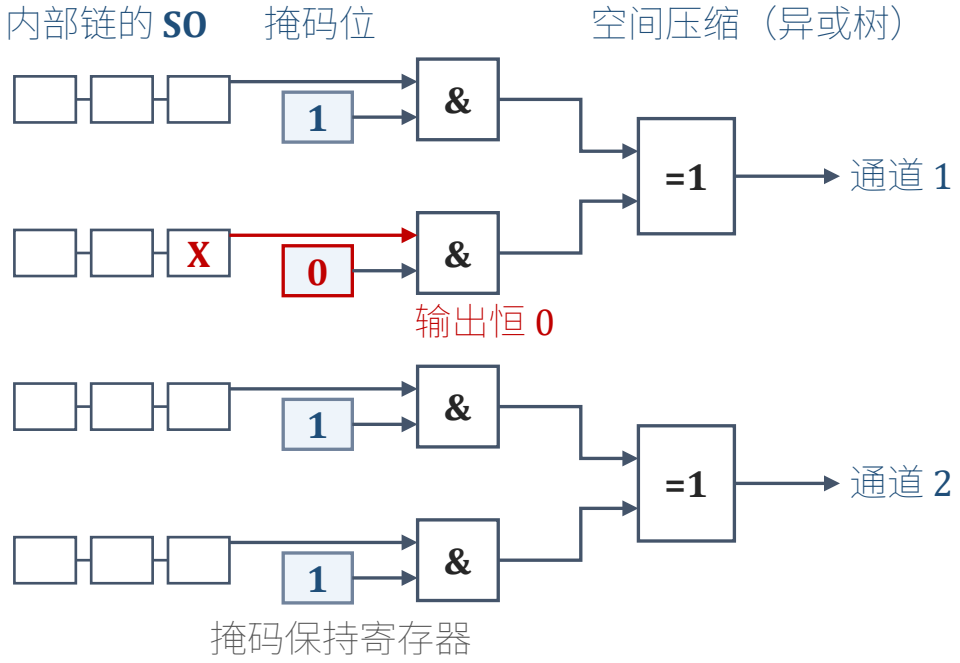
捕获后压缩响应时，要保证故障效应不被遮蔽、X 不污染响应。

X 的影响：异或压缩后，一个 X 会掩盖同通道其他链的响应

屏蔽：掩码位为 0 的链不进入异或树

掩码：随 pattern 移入，装完后并行转存到保持寄存器

X 源：黑盒输出、未初始化单元、跨时钟域路径

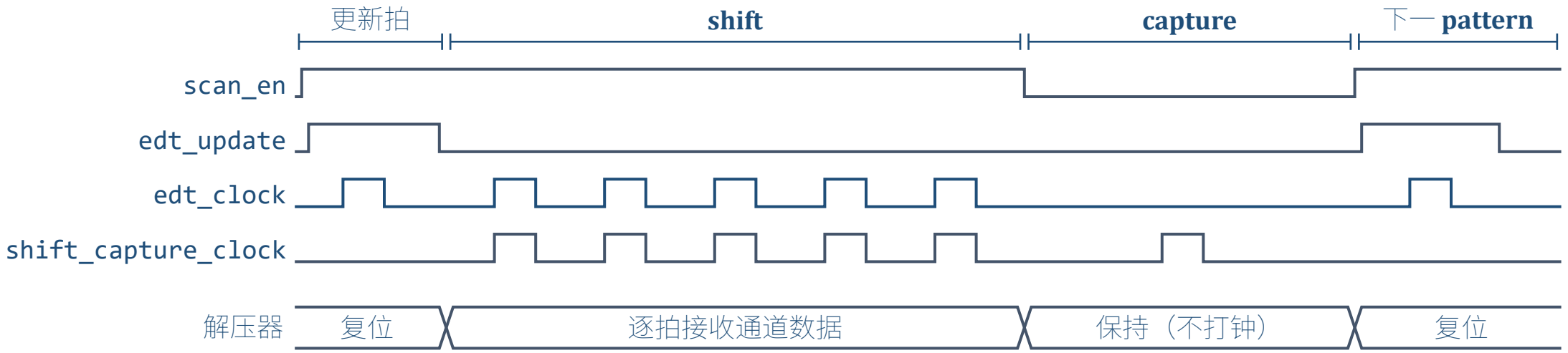


掩码位为 0：该链经与门后恒为 0，X 进不了异或树，不影响同通道其他链

→ 每个 pattern 装载前怎样复位 EDT?

edt_update 为 1 时打一拍 edt_clock，复位 EDT

每个 pattern 装载前都要复位 EDT。



更新拍：`edt_update = 1`，打一拍 `edt_clock` 复位解压器

shift：`edt_clock` 与扫描钟同步，通道数据逐拍进入解压器

capture：`edt_clock` 须为 0；`edt_update` 可不约束（不打钟，不影响 EDT）

→ 不压缩时怎样直接 access 内部链?

压缩旁路：通道直接接内部链，用于调试与特殊 **pattern**

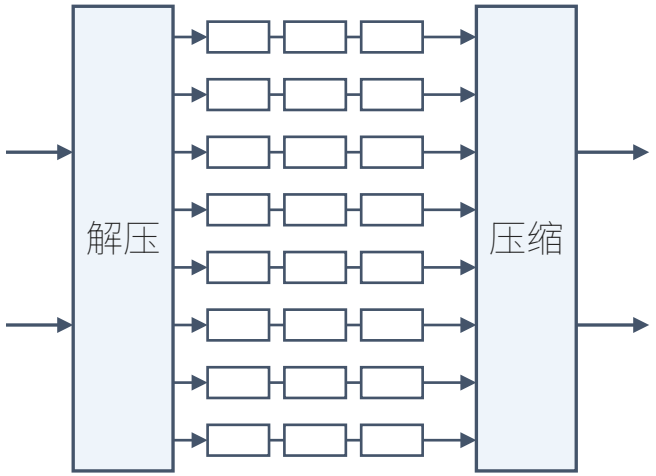
旁路模式用于未压缩 pattern、调试，以及其他 ATPG 工具生成的 pattern。

切换：`edt_bypass = 1` 时，
解压器、压缩器与掩码全部绕开

默认旁路：内部链串成
与通道数相同的几条长链

单链旁路：全部内部链串成一条，
用于通道受限的测试设备

压缩模式：**2** 通道 → **8** 条短链



每个 pattern 移位 ≈ 短链长度 + 初始化

旁路模式：每通道接 **4** 条链串成的长链



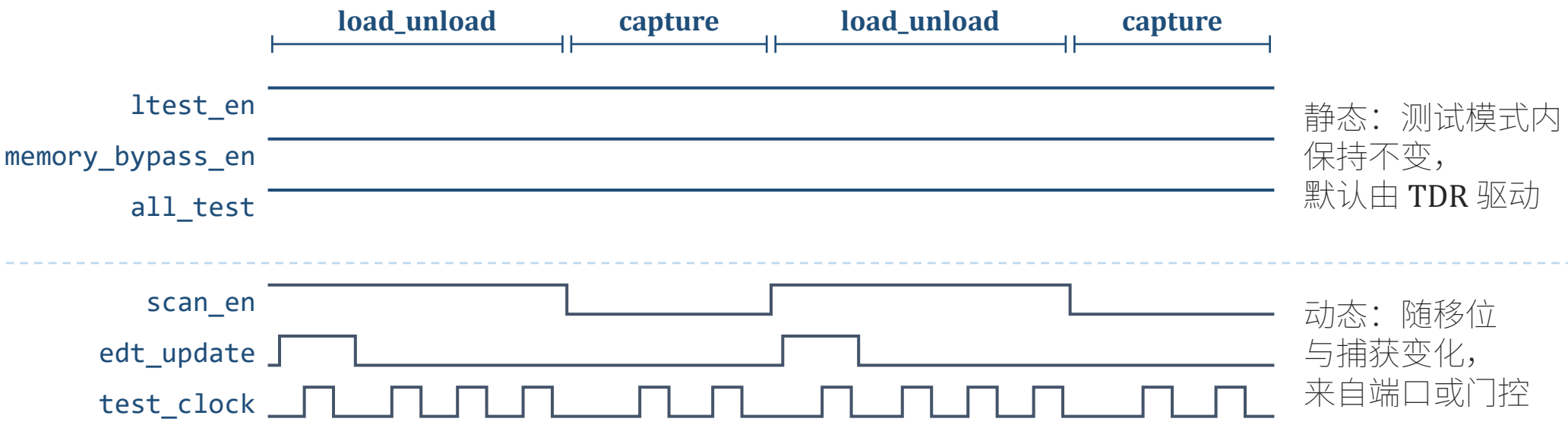
链与链之间按需插 lockup
每个 pattern 移位 = 4 条链长度之和

旁路时 EDT 逻辑不参与，内部链首尾相接直通通道；链变长，移位拍数随之增加，量产一般不用。

→ 静态信号与动态信号有何不同?

静态信号与动态信号：前者在模式内不变，后者随移位与捕获变化

区分两类信号，才能决定每个信号由 TDR 驱动，还是要由端口或门控提供。



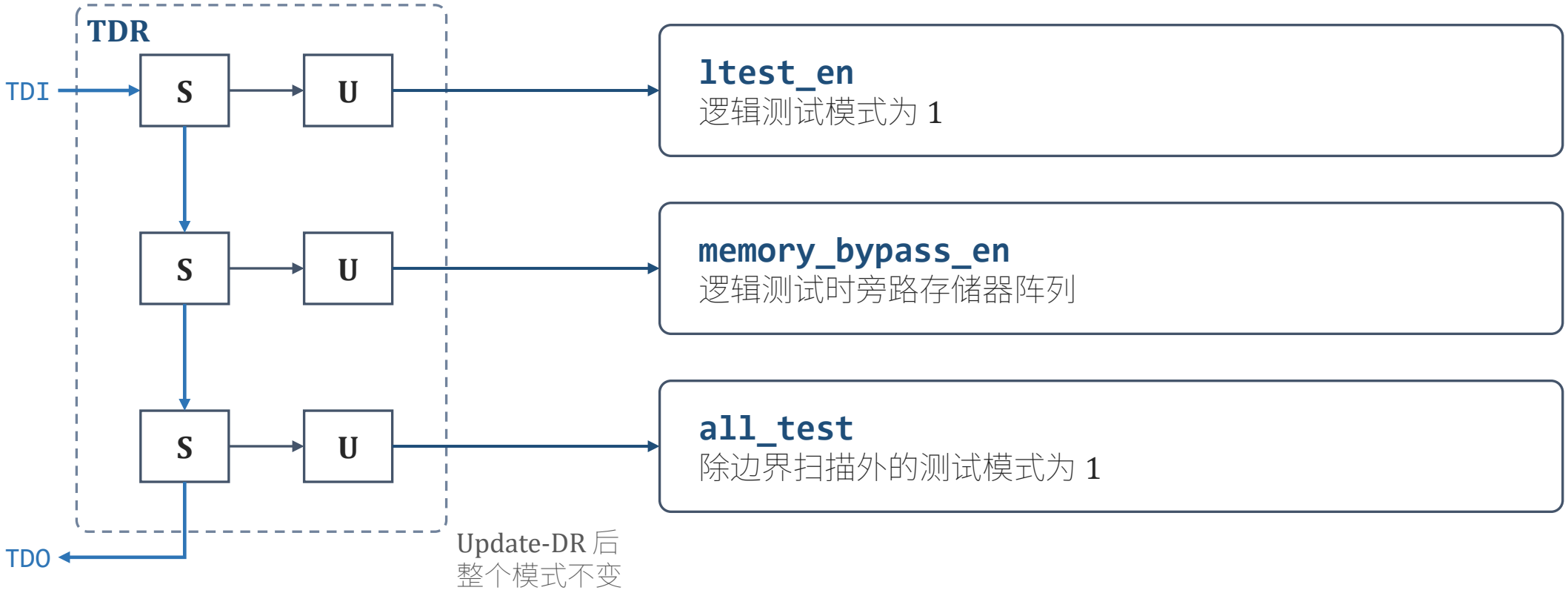
信号来源：指定已有节点、由已有信号派生，或由工具建在 TDR 上（静态信号默认建在 TDR）

有 **SSN** 时：动态信号由 ScanHost 提供（见附录 SSN）

→ 三个关键静态位各管什么？

三个关键静态位：ltest_en、memory_bypass_en、all_test

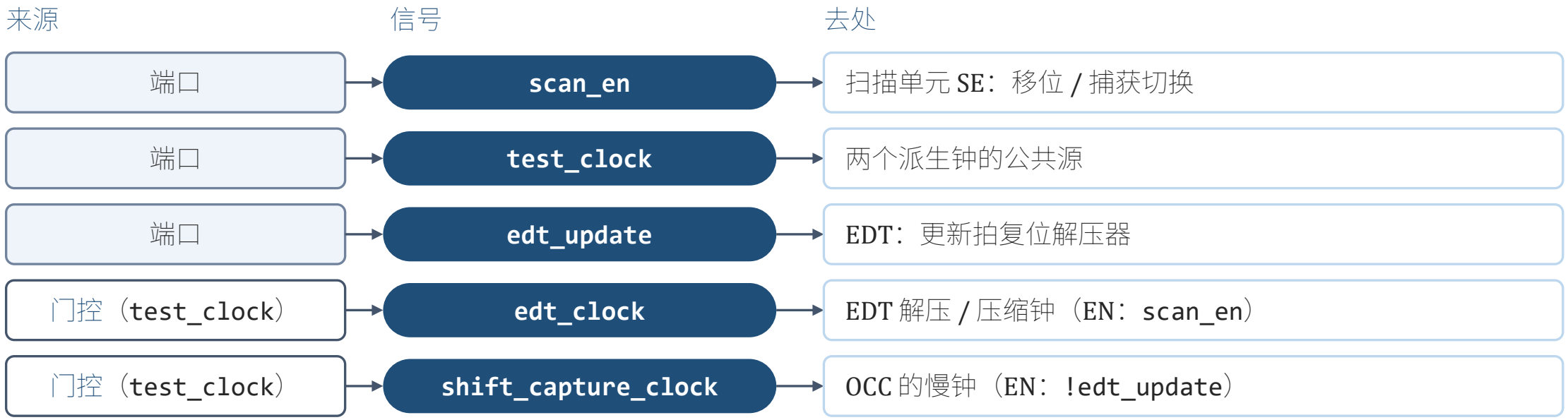
功能复位不能代替这三个静态位。



→ 五个动态信号各管什么?

五个关键动态信号：来自端口，或由端口信号门控派生

三个信号直接来自端口，两个时钟由 test_clock 门控派生，各自带使能条件。



移位时 edt_clock 与扫描钟一起打；捕获期 edt_clock 必须为 0。
动态信号来自端口或由端口信号门控派生，不能只写在约束里。

时序关系见第 45 页（test_clock）与第 50 页（edt_update）

→ 哪些对象要排除在扫描与故障之外？

排除对象须写明范围与理由，并记入对应清单

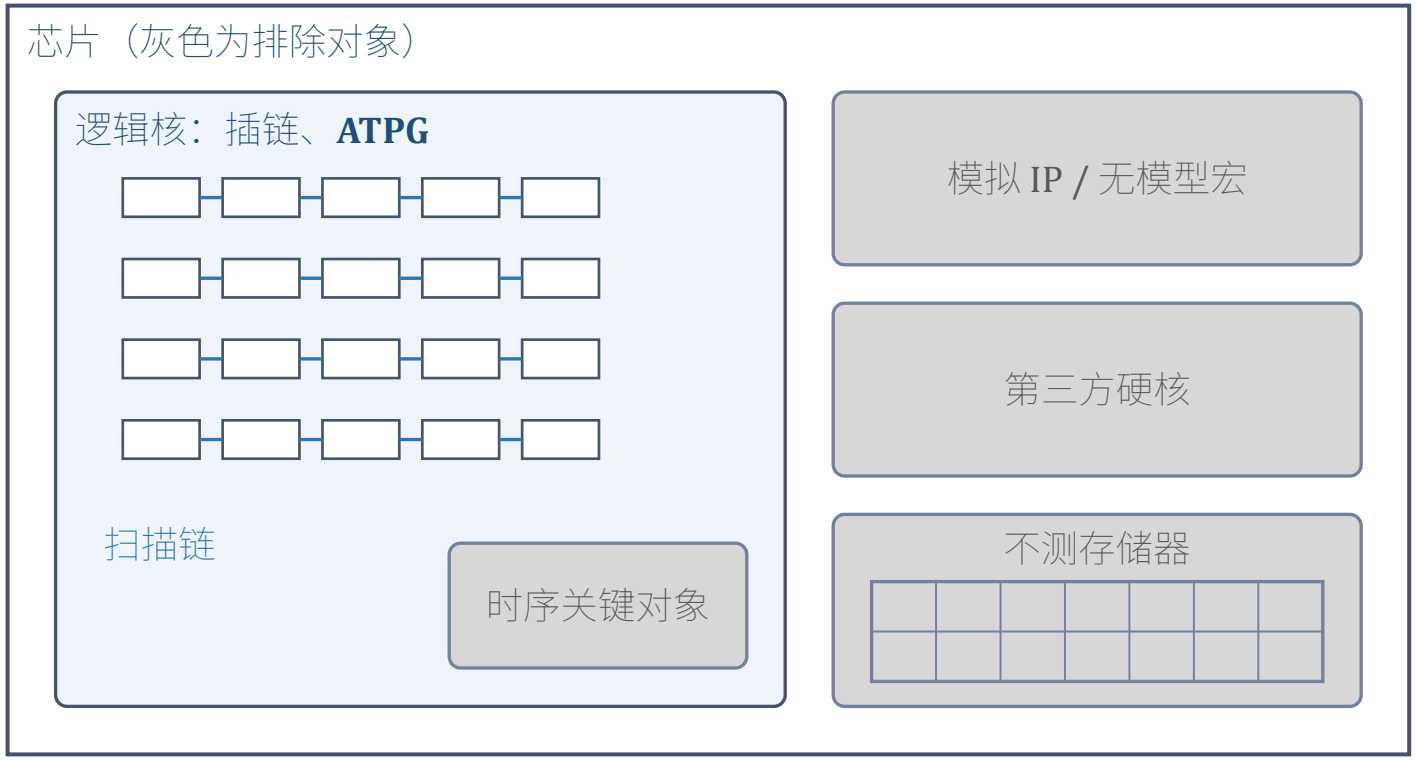
不是所有实例都该进扫描链或故障表。

模拟模块、无模型宏：
没有可供 ATPG 使用的门级模型

第三方硬核：
不可修改，按供应商方案测试

时序关键对象：
插入扫描会破坏时序或功能

不测的存储器：
按设计决策不测，须写明理由



→ 没有内部模型的模块怎样处理?

黑盒只保留端口行为，输出可能是 X 源

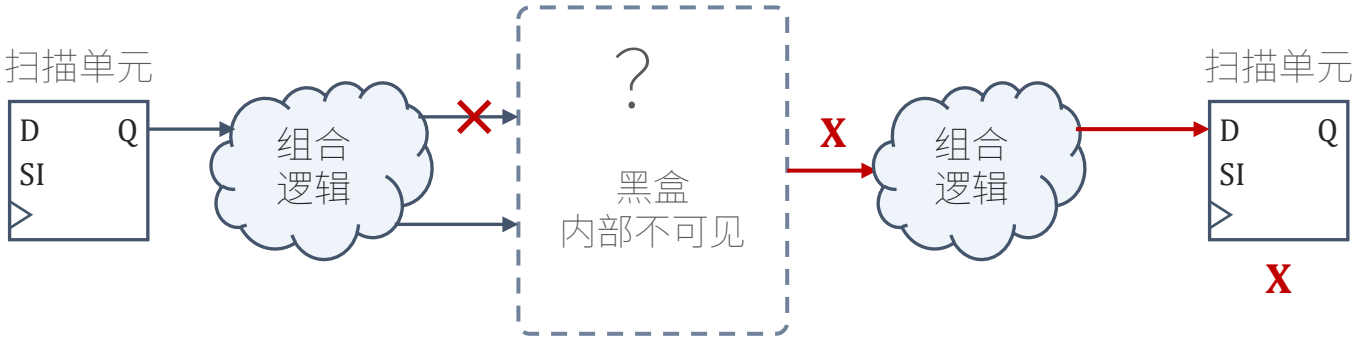
黑盒（black box）：没有内部模型时，只保留端口行为（含输出假定值）。

识别：`add_black_boxes -auto`
把未定义的模型设为黑盒

输出：默认驱动 X，
可用同一命令改为指定值

输入：传到黑盒输入的故障
记为 AU（AU.BB）

X 风险：X 被下游扫描单元捕获，
需与压缩、掩码一起处理



故障效应到黑盒输入为止：记为 AU

捕获到 X：需屏蔽

黑盒输出默认为 X：下游一旦捕获，响应中就含 X；
需为黑盒指定输出值，或用掩码屏蔽。

AU：ATPG untestable（ATPG 不可测）；AU.BB 为其中因黑盒而测不到的一类

→ 四类排除清单各管什么？

四类排除清单各有依据，应分别记录

同一份排除决策分别写入四份清单，供 07 插链、08 ATPG 与 10 STA 使用。



插链排除了某实例，ATPG 不一定知道，反之亦然；四份清单应出自同一份设计决策。

→ 带 wrapper 的核交给上层时，怎样精简网表？

灰盒（**graybox**）是层次化补充，主路径为平坦流程

带 wrapper 的核交给上层时，不必给出完整的核网表。

完整核：上层读入全部网表



灰盒：只留交界部分

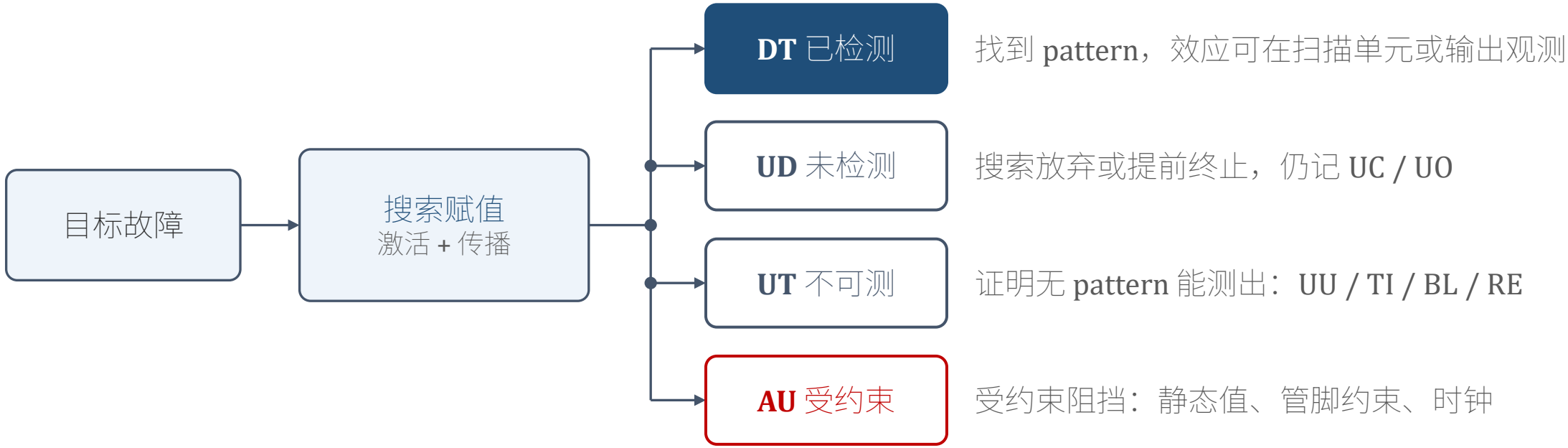


保留：外部模式逻辑（含 wrapper 链）与上层测试仍需的 IJTAG
目的：上层读入更快，仍能测试核与核外逻辑的交界

→ ATPG 怎样生成 pattern 并给故障分类?

ATPG：为目标故障搜索赋值，并给每个故障分类

ATPG 为目标故障寻找激活与传播条件，生成测试 pattern。



AU 可重置：`reset_au_faults` 把 AU 改回 UC，交给工具重新分析

读覆盖率：回到这些类别，不能只看一个百分比

ATPG：Automatic Test Pattern Generation

→ 故障分类有哪几级？

故障分类：同一层级只进一类

同一层级能进多类时，只归入表中靠前的一类；表按分类层级的先后排列。

类别	缩写与子类	含义
可测（**TE**）	DT、PD、AU、UD	不能证明为不可测的故障；test coverage 的分母
已检测（DT）	DS / DI（路径延时另有 DR）	检测成功
可能检测（PD）	PU / PT	默认按 50% 计入
ATPG 不可测（AU）	如 AU.BB（黑盒）	受约束或黑盒阻挡
未检测（UD）	UC / UO	控制不够 / 观察不够
不可测（**UT**）	UU / TI / BL / RE	未用、绑定、阻塞、冗余

→ 覆盖率的分母与分子各算什么？



三种覆盖率：通常看 **test coverage**（分母不含 **UT**）

三种覆盖率都由故障类计数得到，差别在分子、分母各计入哪些类。

test coverage	(#DT + #PD × credit) / #testable	默认 credit 50%
fault coverage	(#DT + #PD × credit) / #full	分母含 UT，数值更低
ATPG effectiveness	(#DT + #UT + #AU + #PU + #PT × credit) / #full	能测出、或证明在当前约束下测不出的故障比例

test coverage 与 fault coverage 分母不同；fault coverage 与 ATPG effectiveness 分子不同。
覆盖率高仍可能有结构问题：先看分类，AU / UC / UO / UU 各自指向不同的补法。

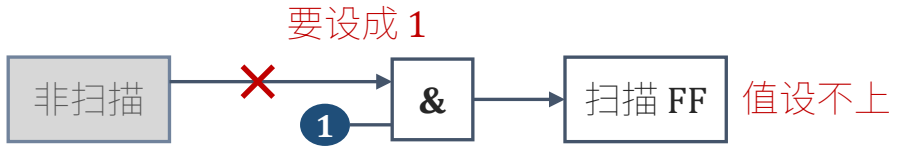
→ UC、UO、UU、TI 分别从哪里补？



UC、UO 靠加控制与观察，UU、TI 要查网表或模型

报告里的数字，要回到可控与可观来读。

UC
控制不够



原因：所需值设不上
补法：加控制点，或改为扫描单元

UO
观察不够



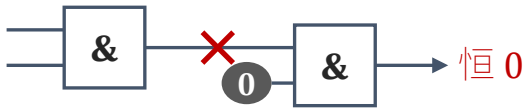
原因：效应传不到观测点
补法：加观察点，或接入扫描链

UU / TI
未用 / 绑定



原因：结构上本就测不出
补法：核对网表或单元模型

BL / RE
阻塞 / 冗余



RE：G = D + ¬D 恒为 1，
D 的 stuck-at-1 无法检出

原因：逻辑阻塞或冗余
补法：改 pattern 无效，需改逻辑

UU：Unused；TI：Tied；BL：Blocked；RE：Redundant

→ pattern 有哪几种形态？

pattern 形态：scan / chain × 串行 / 并行

用途、装载方式、文件格式是三个独立维度；并行通过不能代替串行移位。

	串行装载	并行装载
scan pattern	真实移位，反映链的实际行为	快速检查 pattern，不验证移位
chain pattern	专门检验链完整性	只作对照，不能替代串行

文件格式		
Verilog testbench	STIL	WGL

STIL：Standard Test Interface Language（IEEE 1450）；WGL：Waveform Generation Language

→ Part 2 的方法彼此怎样依赖？

Part 2 小结：各方法按硬件依赖衔接

把 Part 2 的各块内容按硬件依赖排成一条内容流。

故障与可控可观
故障模型决定 pattern；
扫描补上可控可观

→

扫描单元与链
MUX-DFF 串成链：
移入、捕获、移出

→

TAP / TDR
TCK / TMS 推动状态机，
TDR 位驱动静态信号

→

边界扫描与 IJTAG
边界扫描测互连；
SIB 按需接入 instrument

↓

MBIST
片上算法测阵列，
外部只启动、读结果

→

OCC / EDT
片上生成至速捕获钟；
少通道驱动多短链

→

信号与排除
静态信号与动态信号；
四类排除清单分别记录

→

ATPG 与覆盖率
每个故障只进一类；
通常看 test coverage

依赖是物理的：没有扫描链就没有 ATPG，没有 ICL / PDL 就生成不了 instrument 的 pattern。

→ Part 3：各方法在工程流程中负责哪一步？

03

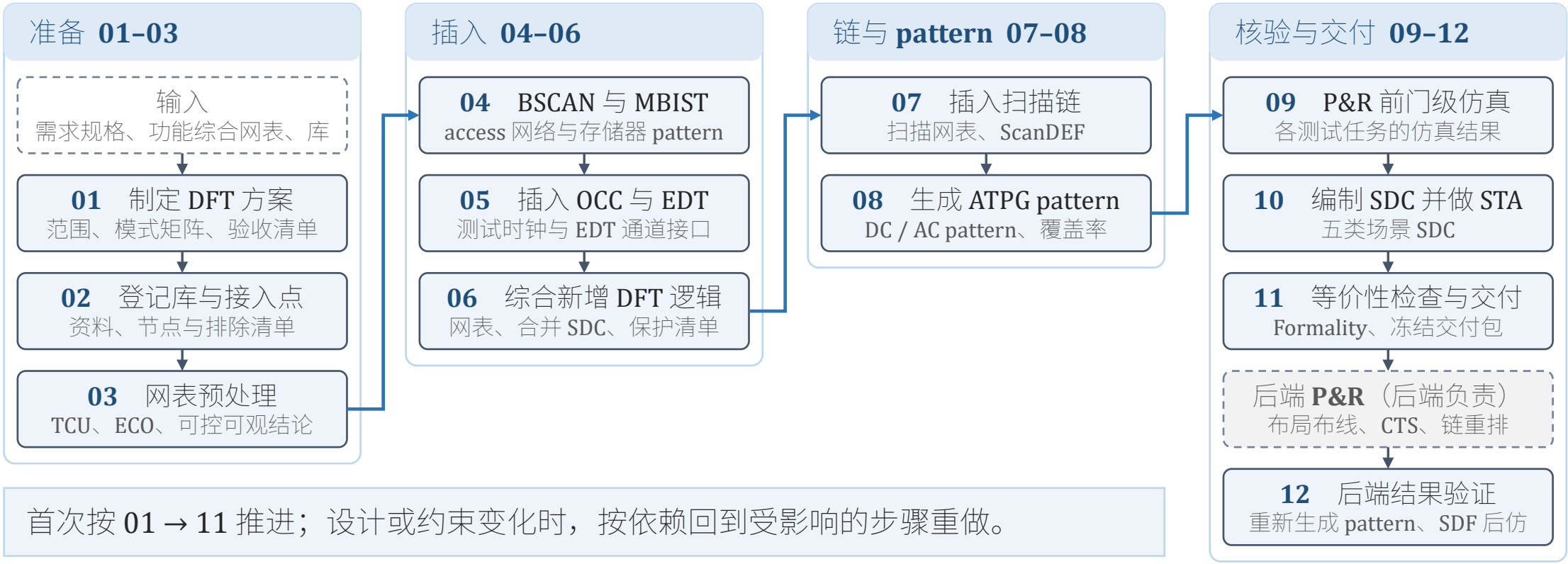
Part 3　工程流程

准备、插入、链与 pattern、核验与交付

各方法在工程流程中负责哪一步？

12 步流程：准备、插入、链与 pattern、核验与交付

01 从需求定义开始，02 起使用功能综合网表；11 交付后端，12 收回后端结果做时序后仿。



首次按 01 → 11 推进；设计或约束变化时，按依赖回到受影响的步骤重做。

→ 每种方法落在哪几步？

方法与 12 步对照：每种方法在哪一步定义、实施和核验

每行从左到右：先在准备阶段定义或登记，在深色步骤实施，再在后续步骤使用与核验。

Part 2 的方法	准备 01–03	插入 04–06	链与 pattern 07–08	核验与交付 09–12
TAP / BSCAN / IJTAG / TDR	01 定 access 架构	**04** 插入 **access** 网络	08 汇集 access 测试	09 / 12 仿真
MBIST	02 登记存储器模型	**04** 插入 **MBIST**	08 汇集已生成 pattern	09 门级仿真
OCC / EDT	02 登记 OCC 接入点	**05** 插入 **OCC、EDT**	07 链接到 EDT	10 各模式 SDC
扫描链 / **lockup** / 测试点	01 定扫描预算	06 收敛成门级网表	**07** 插链、测试点	12 核对链重排
ATPG / 覆盖率	01 定验收覆盖率		**08 DC / AC ATPG**	09 / 12 仿真 pattern
排除清单（非扫描、黑盒等）	**02** 落成对象清单	04 / 05 应用	07 / 08 应用	

深色 = 实施该方法的主要步骤；灰盒 / wrapper 属层次化补充流程，不占主 12 步。

→ 01–03 准备阶段做什么？

01–03：只做准备，测试硬件从 04 才开始插入

准备阶段确定方案、登记资料与接入点，并把网表整到可控可观；不插入任何测试硬件。

01 制定 DFT 方案

输入	产品功能、接口、测试与交付需求
动作	定测试范围、access 架构、时钟架构、模式及进入方式
输出	DFT 方案、模式矩阵、验收清单

02 登记库与接入点

输入	01 的方案、功能综合网表、各类库与模型
动作	清点测试库与存储器模型，登记时钟、复位、OCC 接入点
输出	资料、节点与排除清单（按散列固定，缺项注明）

03 网表预处理

输入	功能综合网表、02 的清单、Pad / I/O 信息
动作	建 TCU 控制层次与缓冲，做准备性 ECO，跑 DRC
输出	预处理网表、DRC 报告、可控可观达标结论

界线：04 起才插入测试硬件；03 只建 TCU 骨架、做准备性 ECO，不插 TAP、BSCAN、MBIST，也不把寄存器换成扫描单元。

TCU：Test Control Unit（片上测试控制单元）；ECO：Engineering Change Order；DRC：Design Rule Check

→ 04 一次插入哪些测试硬件？

04：一次插入 BSCAN 与 MBIST，再生成 access 与存储器 pattern

插入与 pattern 生成在同一遍完成：先插硬件、提取 ICL，再按提取结果生成 pattern。

一次插入：TAP、BSCAN、SIB、TDR 与 MBIST 在同一遍插入

生成次序：先插硬件、提取 ICL，再按 ICL 生成 pattern

pattern：`ICLNetwork` 验 access，`MemoryBist_P1` 等测存储器



为什么同一遍：BSCAN 与 MBIST 接在同一个 IJTAG access 网络上；分两遍会多一个设计版本和一次增量综合。

对应命令：`process_dft_specification` → `extract_icl` → `create_patterns_specification`

→ 05 插入哪些逻辑测试硬件？

05：先插入 OCC 与 EDT，07 的扫描链才有接口可接

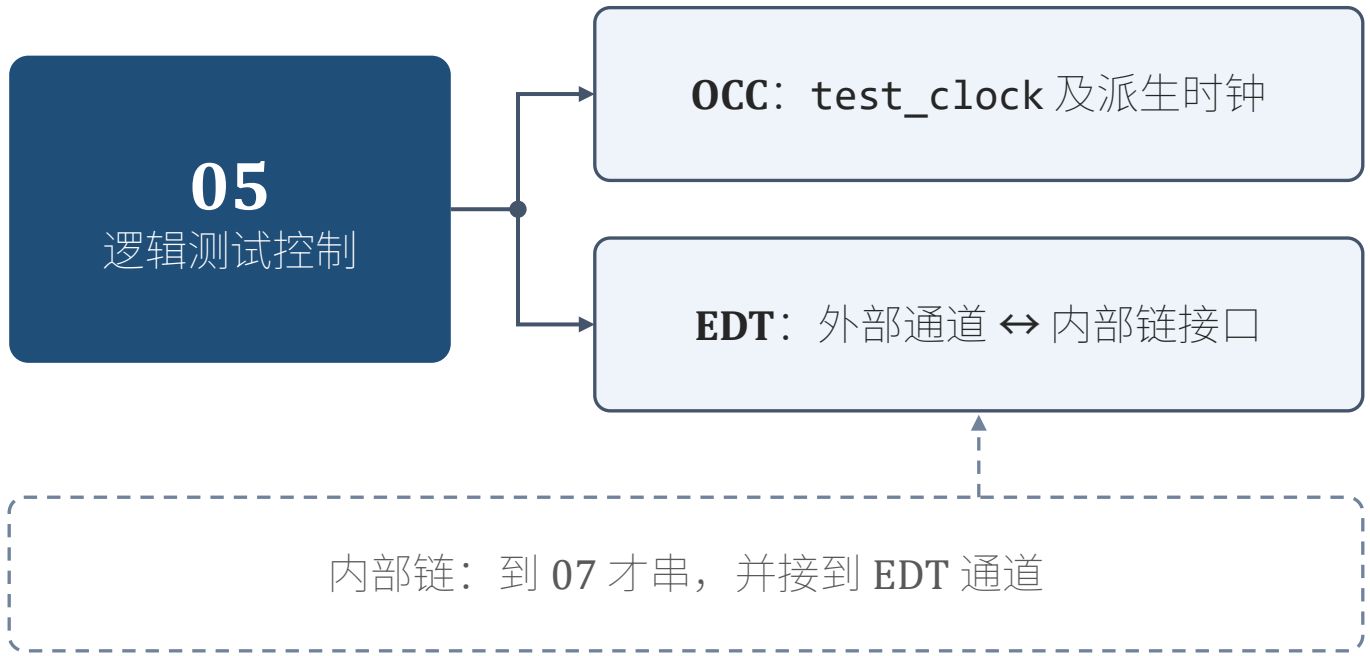
05 插入逻辑测试所需的时钟、控制与压缩硬件；扫描链到 07 才串。

时钟：插入 OCC，
建立测试时钟与捕获控制

控制：接入 scan_en、edt_update
与测试模式信号

压缩：插入 EDT，
确定外部通道与内部链接口

CTS：功能时钟树仍由后端完成



→ 新增的 DFT 逻辑怎样综合与合并?

06：只综合新增的 DFT RTL，再与原网表链接成门级网表

07 要在纯门级网表上把寄存器换成扫描单元，所以混合设计在 06 收敛成门级。



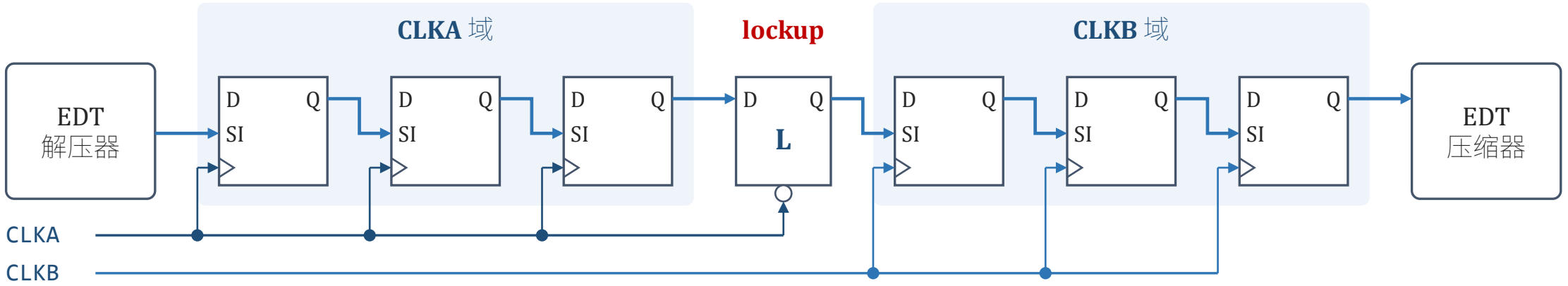
为什么在 06：替换扫描单元须在门级上做；
04 / 05 在 `-rtl` 下插入，只写综合脚本

保护：Tessent SDC 给出的实例
与工程关键 cell 均不被优化

→ 07 插链时还做什么？

07：插入扫描链，按需在同一遍加入测试点

07 在 06 的门级网表上把寄存器换成扫描单元并串链；跨时钟域处由工具自动加 lockup。



lockup 在 CLKA 为低时透明：数据晚半拍交给 CLKB 域

lockup 从哪来：插链时由工具自动插入，
位于时钟或时钟沿不同的相邻单元之间；默认用锁存器

链两端：链首、链尾与 EDT 相接处，
按时钟沿要求也会插入；EDT 逻辑内另有 lockup

测试点：同一阶段两套脚本：`scan` 不插，
`scan_test_points` 同遍插入；做不做由 01 定

产物：扫描网表、ScanDEF、链与单元报告，
供 08 生成 pattern、后端重排链使用

→ 08 生成哪些 pattern?

08：生成逻辑测试 pattern 并统计覆盖率，汇集全部测试任务

测试任务清单按用途列出：每项测什么、为什么测、pattern 在哪一步生成。

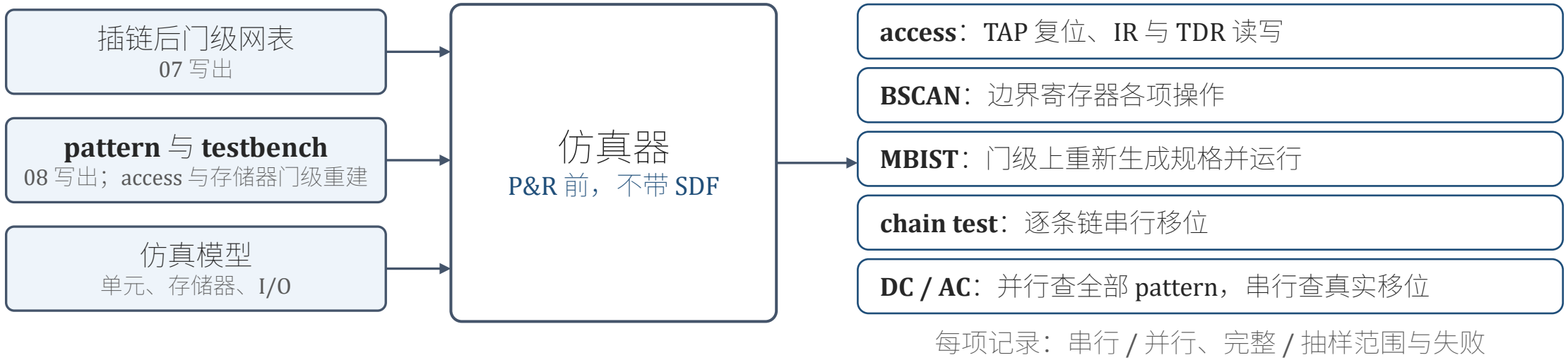
任务	测试对象	目的	pattern 来源
access（**TAP / IJTAG**）	TAP 复位、IR、BYPASS； 每个 TDR 能选中、长度与读写正确	后续配置都经 TAP 写 TDR， access 不通，其余测试都做不了	04：ICLNetwork、 JtagTapPatterns
BSCAN（**BSR**）	边界单元与管脚对应、输入采样、 输出控制、高阻与 clamp	板级工具按 BSDL 操作这些单元， 先证明各项边界扫描操作可用	04：JtagBscanPatterns
MBIST	控制器启动、算法运行、 Go / NoGo 与状态读取	确认每个存储器都被测到， 结果能经 TAP 读回	04：MemoryBist_P1 等
扫描链（**chain**）	每条链的移位连通与长度	链不通，所有 scan pattern 都无效，所以先验证	08：chain pattern
DC（**stuck-at**）	stuck-at 等静态故障， 慢速捕获	统计 DC 覆盖率， 检出静态缺陷	08：DC ATPG 读 SDC 的 hold 例外
AC（**transition**）	transition、path delay 等 延时故障，OCC 至速捕获	统计 AC 覆盖率， 检出小延时缺陷	08：AC ATPG 读 SDC 全部例外

08 只生成逻辑测试 pattern；存储器与 access pattern 已在 04 生成，这里只汇集。DC 与 AC 分开生成，AU 不互相沿用。

→ 09 仿真验证什么？

09：用插链后的门级网表做 P&R 前仿真

综合与插链之后、交给后端之前，在门级网表上逐项仿真各测试任务的 pattern。



串行与并行：并行装载快速核对 pattern；chain test 与串行 pattern 验证真实移位，两者不能互相代替

网表一致性：仿真用的就是 07 写出的网表，按散列核对；09 不做网表比对，功能等价性在 11 由 Formality 检查

不代替：功能 RTL 仿真不能代替门级仿真；抽样结果不外推为全量制造测试

→ 10、11 怎样核验约束并交付？

10–11：编制 SDC 并用 STA 核验，做等价性检查后交付后端

10 出五类场景 SDC 并用 PrimeTime 核验；11 做等价性比对，冻结交付包交后端。

10　SDC 编制与 STA 核验

产出：五类场景 SDC 与加载说明

另附：SDC 之外的设置，每类一份调用脚本

核验：PrimeTime 查约束覆盖与 setup / hold

边界：保护清单、CTS 排除点归 06；不做布局布线

11　等价性检查与交付

后处理：只做有依据的交付前修改

等价性：Formality，参考为原始功能网表

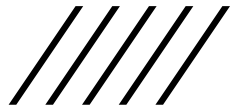
交付包：网表、ScanDEF、SDC 与脚本、保护清单

一致性：交付网表即 09 仿真所用的那份

交付时序：场景 SDC 与相关脚本随 11 的交付包先交后端；P&R 期间按后端反馈多轮更新，直到收敛。收敛后后端交回网表与 SDF，由 DFT 在 12 重新生成 pattern、做反标仿真。

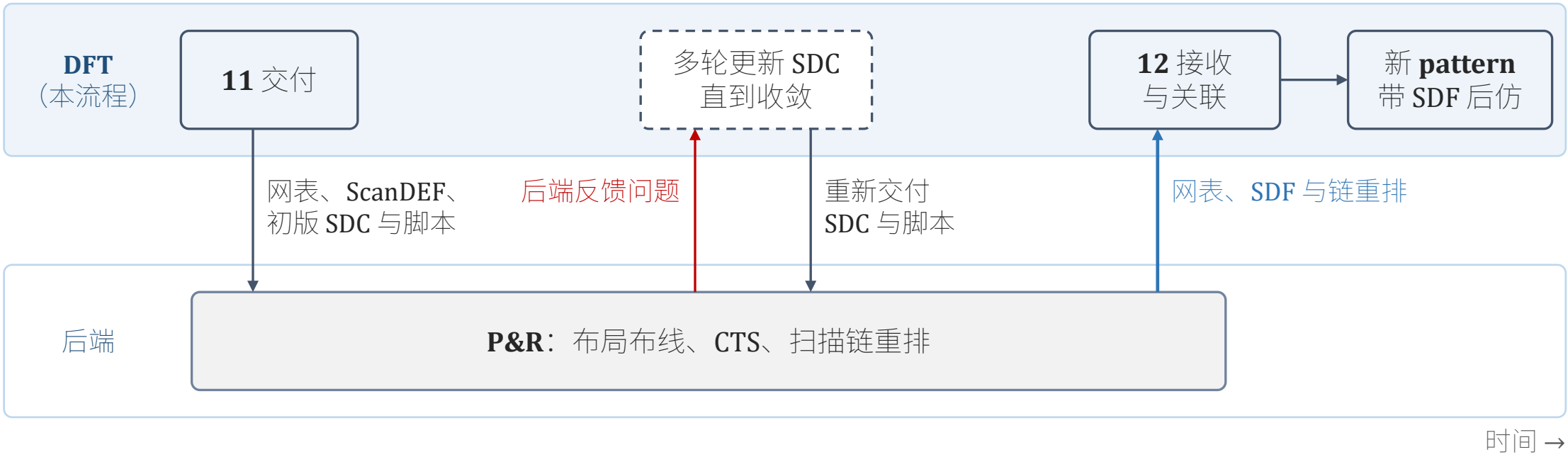
SDC：Synopsys Design Constraints；STA：Static Timing Analysis；SDF：Standard Delay Format

→ 后端结果回来后做什么？



12：接收后端网表与 SDF，重新生成 pattern 并做时序后仿

P&R 期间 SDC 迭代到收敛；收敛后后端交回网表与 SDF，由 DFT 重新生成 pattern。



12 做：独立保存返回版本，关联 ICL / TCD / 扫描模型，核对链重排，重新生成 pattern，用 SDF 做时序后仿

12 不做：收敛后不再给后端重出 SDC；pattern 由 DFT 生成，后端只做布局布线

CTS：Clock Tree Synthesis；SDF：Standard Delay Format（反标仿真的延时数据）；TCD：Tessent Core Description

→ 12 步的主路径做什么、不做什么？

12 步的职责边界：主路径做什么，不做什么

左栏是主路径各阶段的实际动作；右栏的做法默认不做，需要时另加。

主路径做

- 01–03：定方案、登记资料、预处理网表
- 04–06：插入测试硬件并综合新增逻辑
- 07–08：插扫描链，生成 DC / AC pattern
- 09–12：P&R 前仿真、SDC 与交付、后端验证

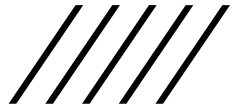
主路径不做

- BISR、SSN、LBIST：按需加入，默认不做
- 层次化 graybox：只作补充流程
- 03 不插入任何测试硬件
- 08 不生成存储器 pattern（04 已生成）

次序：定方案 → 登记资料 → 预处理网表 → 插 BSCAN / MBIST → 插 OCC / EDT → 综合 → 插链 → ATPG → P&R 前仿真 → SDC 与交付 → 后端结果验证

BISR：Built-In Self-Repair（存储器内建自修复）；SSN：Streaming Scan Network；LBIST：Logic BIST（逻辑内建自测试）

→ 附录：只点名、不作主路径的做法



附录

Appendix

只点名、不作主路径的做法

BISR：存储器内建自修复（Built-In Self-Repair）

SSN：流式扫描网络（Streaming Scan Network）

LBIST：逻辑内建自测试（Logic BIST）

测试点与 X-bounding：补可控可观，阻断未知值

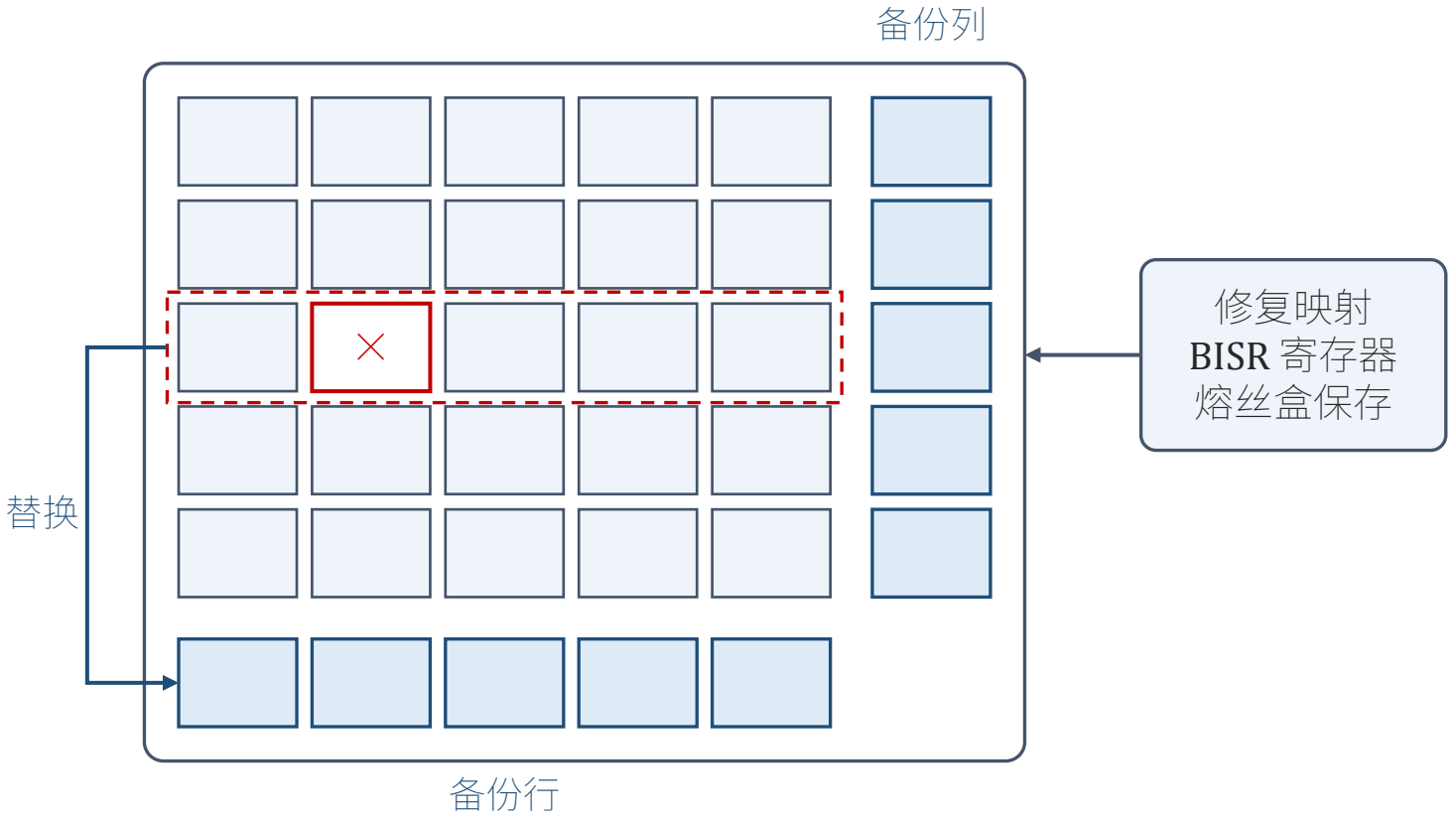
BISR：用备份行列替换故障位置（可选）

备份单元：存储器自带
备份行 / 列

修复分析：MBIST 运行时，
由 BIRA 算出修复值

保存：BISR 寄存器串成链，
由 BISR 控制器写入熔丝盒

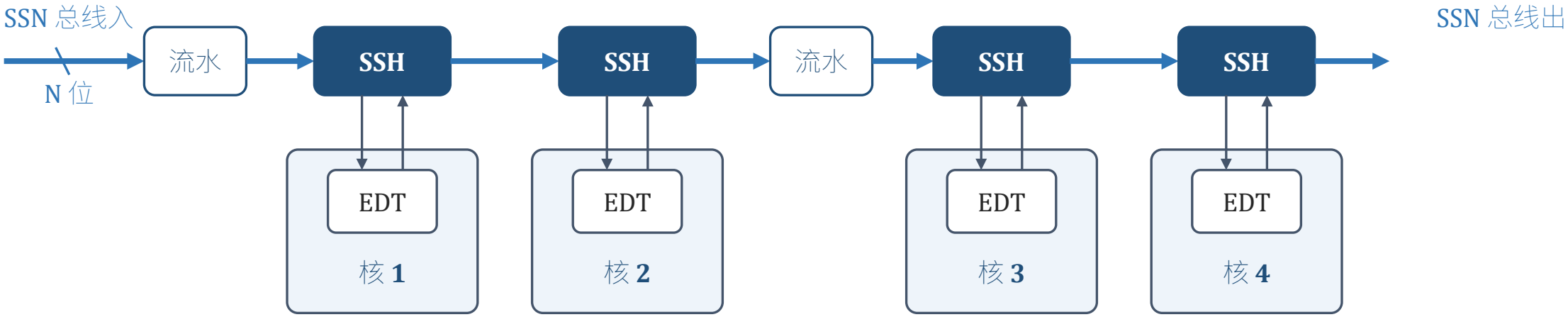
硬 / 软修复：熔丝永久保存，
或每次上电重新装载



BISR：Built-In Self-Repair（存储器内建自修复）；BIRA：Built-In Redundancy Analysis（冗余分析）

SSN：用总线分发核级扫描数据（未作主路径）

SSN 用一条总线把扫描数据分发给各核的 ScanHost，核级压缩配置与顶层管脚数解耦。



EDT 通道直接接本核的 ScanHost；总线经过所有 SSH，跨区处加流水级

逐个穿过：SSN 总线依次经过每个核的 ScanHost（SSH）

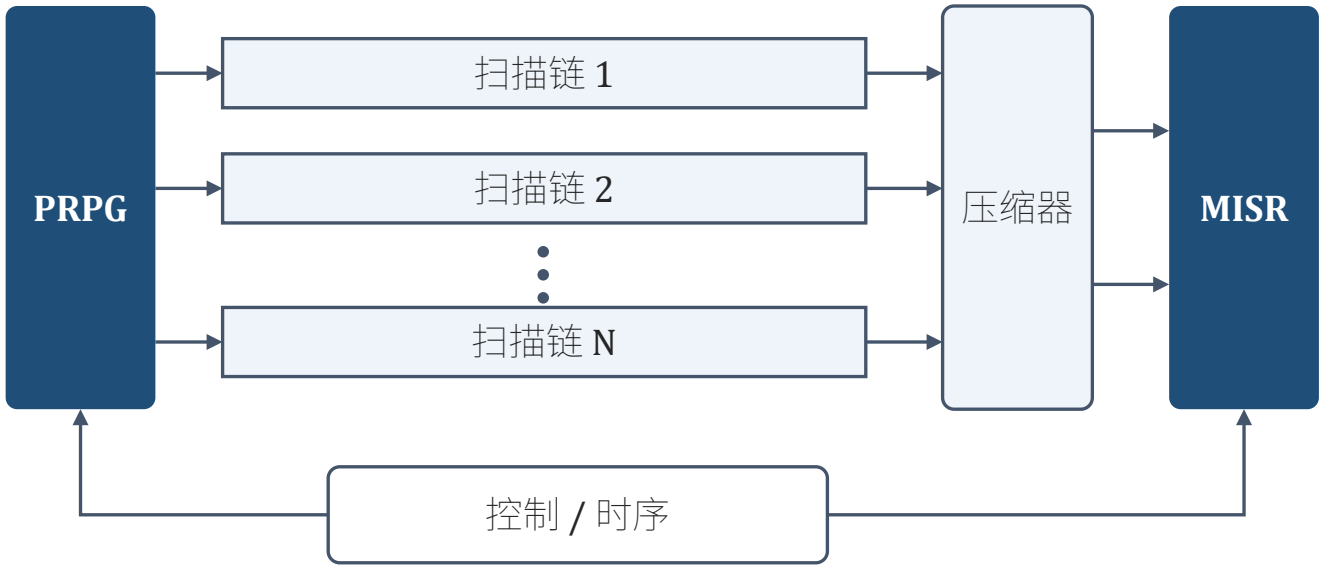
解耦：核级压缩按核自己定，与顶层管脚数无关

选核：哪些核同时测，可推迟到生成 pattern 时再定

SSN：Streaming Scan Network（流式扫描网络）

LBIST：PRPG 生成激励，MISR 压缩响应

LBIST 在片上产生伪随机 pattern、压缩响应，外部只启动测试并读回签名。



PRPG：伪随机 pattern 发生器，pattern 移入内部链

MISR：把移出的响应压成多位签名

控制 / 时序：安排 PRPG、MISR 与扫描链的动作

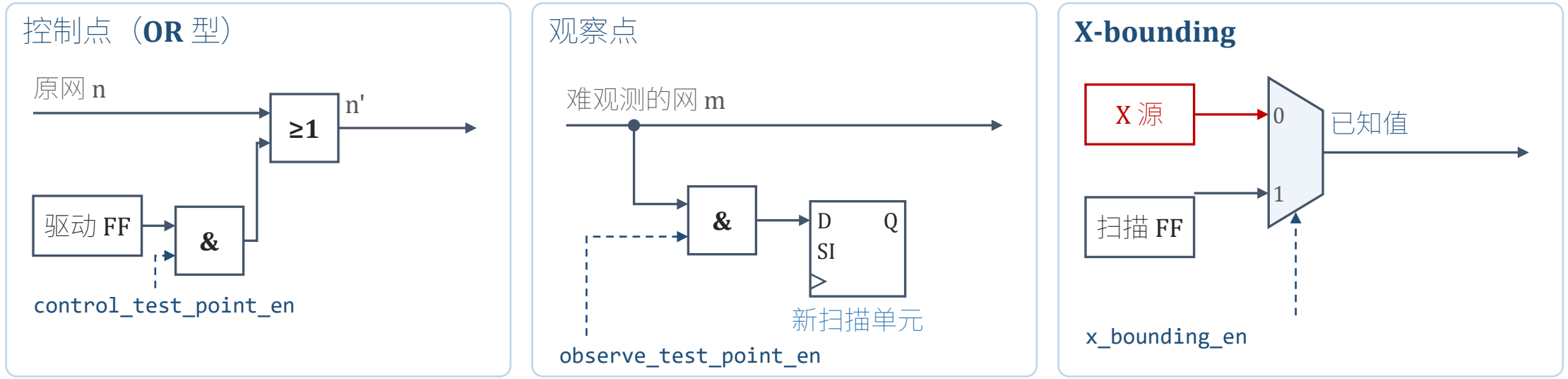
① PRPG、MISR 扫描初始化 → ② 多轮：装链 → 捕获 → 移出并压入 MISR → ③ 移出签名，与期望值比较

PRPG 与 MISR 在 hybrid EDT 控制器内，可与 OCC、EDT 在 05 同一次插入；随机 pattern 的故障仿真在 08 按需进行

LBIST：Logic Built-In Self-Test（逻辑内建自测试）；
PRPG：Pseudo-Random Pattern Generator；MISR：Multiple-Input Signature Register

测试点提高可控与可观，**X-bounding** 阻断未知值

控制点、观察点补可控与可观；X-bounding 在 X 源后面把未知值挡住。



控制点：强制网取 1，提高可控性

观察点：用新扫描单元捕获网 m，提高可观性

X-bounding：以已知值替代 X，保护压缩

本流程：测试点按需在 07 与插链同遍加入

04

Part 4　常见错误与排查

读报告、找原因、查手册

报错了怎么读、怎么修、去手册哪里找例子?

报错集中在 **04、05**：插入前的 **DRC** 先查时钟与复位

04、05 插入前的 DRC 追时钟与复位，报错最集中；07、08 查链与捕获，报错较少。




```
// Error: The clock 'clk_REF' must be defined with a period.
//        The clock is used by memory_bist. (DFT_C5-1)
```

等级：Error 不过，工具停在 setup　规则号：决定查哪一节　序号：第几处

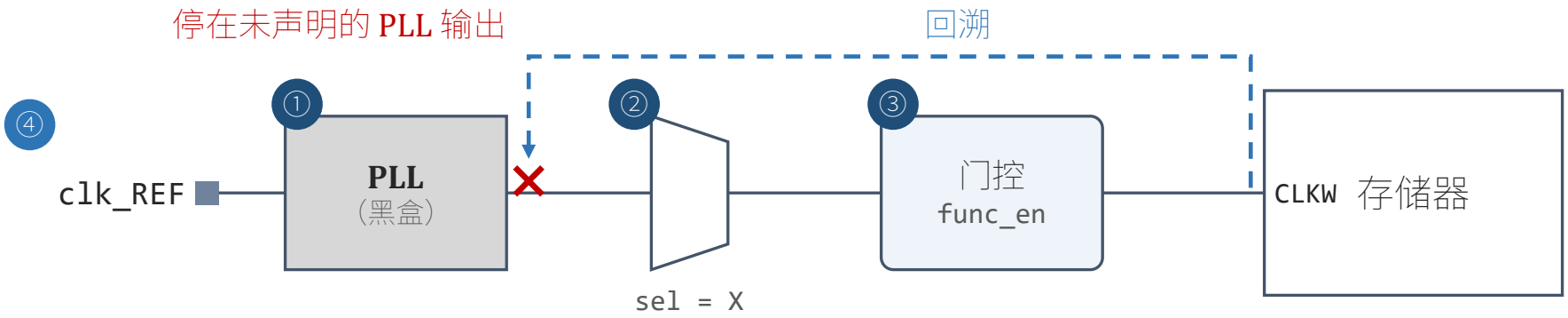
开关：`-memory_test on` 查存储器时钟；`-logic_test on` 查扫描单元的时钟、门控与复位

顺序：03 预查 DFT_C6、C9；04 新插的 MBIST 触发器到 05 才被 DFT_C6 查到

→ 04 的存储器时钟为什么追不到？

04：存储器时钟要追到声明过的时钟源（DFT_C1、DFT_C5）

DFT_C1 从存储器时钟脚往回追，停在未声明的点就报错；DFT_C5 要时钟源有周期。




```
// Error: The memory clock pin '…/mem1/CLKW' (23.28) source tracing stopped at pin 'pll_Inst/pll_out_2' … (DFT_C1-1)
```

① 黑盒 **PLL**：`add_clocks <输出> -reference` 片上振荡器用 `-period`

② 选择器：选择端由未初始化逻辑驱动，用 `add_dft_control_points` 加控制

③ 门控：`add_dft_clock_enables` 指到功能使能脚

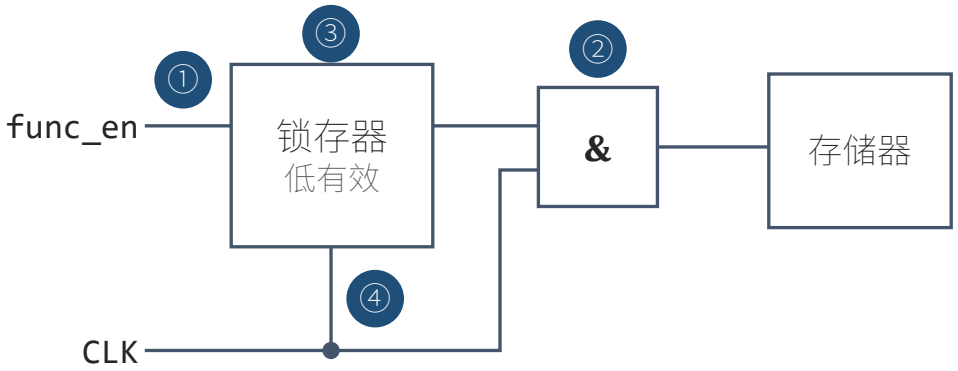
④ **DFT_C5**：`clk_REF` 没有周期，补 `-period`；DFT_C1–C5 的处理方式都不能改

PLL：Phase-Locked Loop（锁相环）；黑盒：只有端口、看不到内部的模块

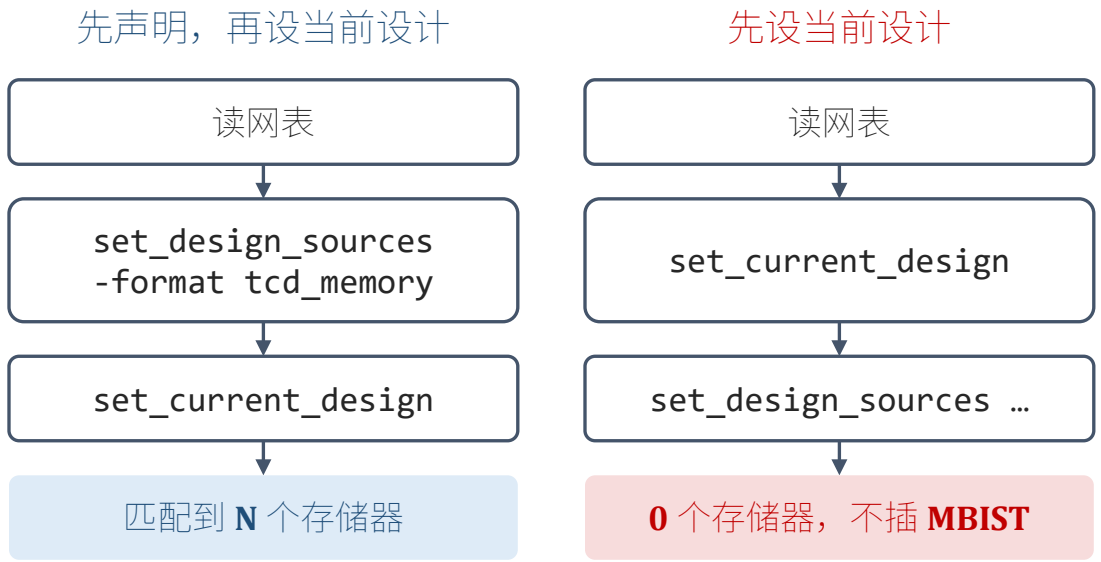
→ 存储器时钟上的门控要满足什么？

04：门控要合规；存储器描述要在设当前设计之前读入

DFT_C7 查存储器时钟上的门控；存储器描述读晚了，工具认不出存储器，也不插 MBIST。



① 功能使能唯一　② 门控是两输入 AND 或 OR
③ 使能路径有锁存器，极性对（AND 配低有效）
④ 锁存器的时钟与门控同源



不合规时：`add_dft_control_points` 加在功能使能上，
把门控关掉，代价是它在逻辑测试里测不到

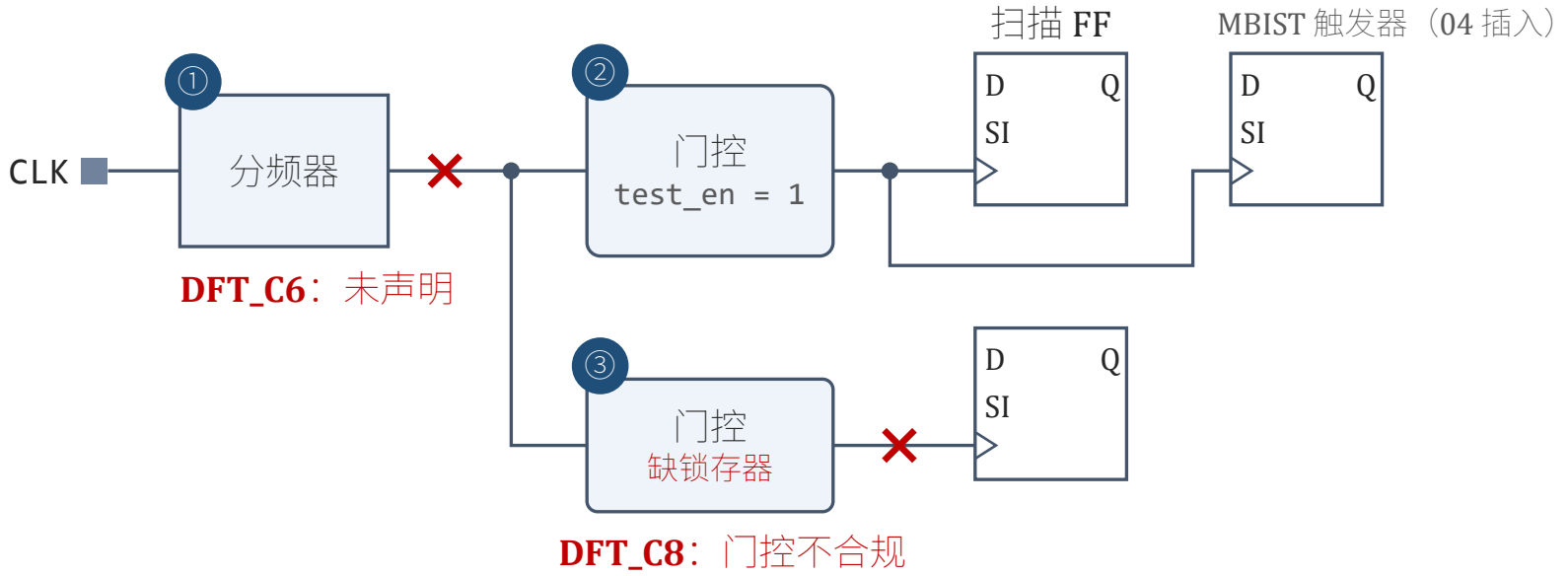
核对：`report_memory_instances` 列出的存储器不是 0 个，
并与 02 登记的存储器清单对得上

TCD：Tessent Core Description（存储器等对象的描述）；功能使能：门控在功能模式下的使能端

→ 05 的扫描单元时钟要满足什么？

05：扫描单元时钟要追得到、门控要合规（DFT_C6、DFT_C8）

开 -logic_test on 后，扫描触发器的时钟要追到声明的时钟，门控也要合规。



① 分频输出：
`add_clocks <输出> -reference CLK`

② 门控挡住：
`add_dft_clock_enables -usage test_en`

③ 门控不合规：
测试使能接 `ltest_en`，
门控常开

降级：DFT_C6、DFT_C8 都可降为 Warning，但时钟问题还在；③ 的代价是功能使能一侧的扇入测不到。

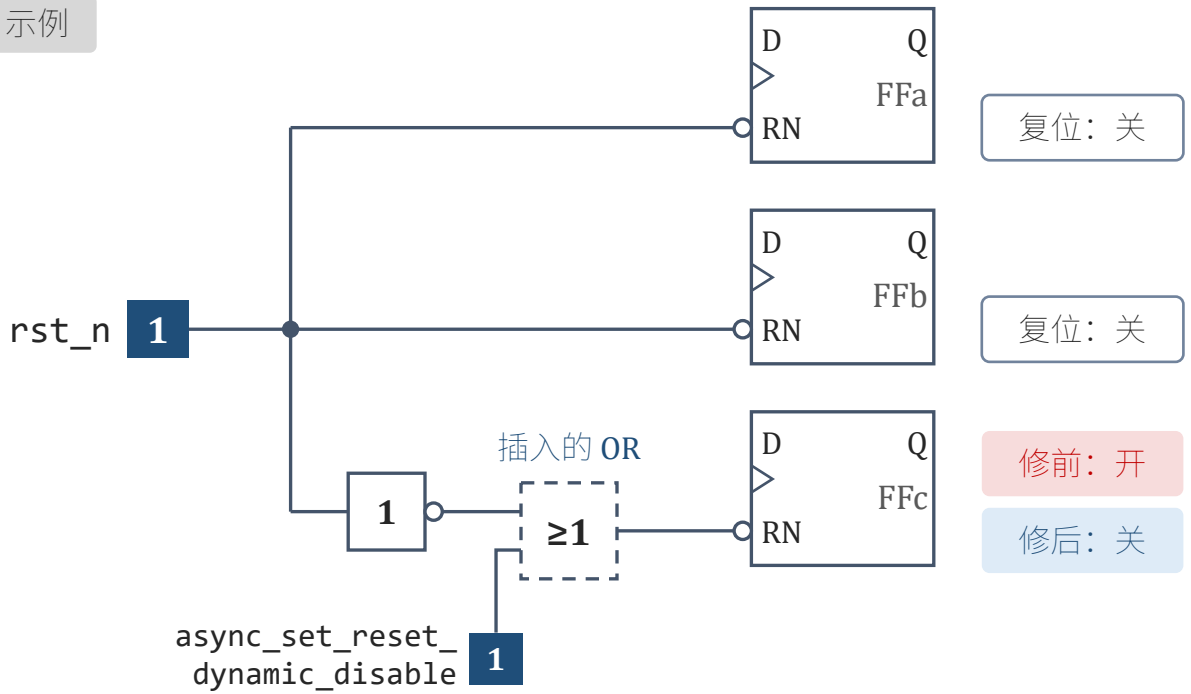
`ltest_en`：逻辑测试期间为 1 的静态信号（`add_dft_control_points <脚> -dft_signal_source_name ltest_en`）

→ 异步复位怎样在移位时关住？

05：每个触发器的异步复位都要在移位时关得住（DFT_C9）

DFT_C9 把复位源设到让多数复位失效的值，再看剩下的复位是否反被打开。

示例




```
When the async_set_reset source rst_n is asserted '1'
2 reset pins on DFFs are forced off but 1 reset pins
on DFF are forced on.
```

→ OCC 的注入点要放在哪里?

第一类：不同极性共用一个源

在少数那一支 `add_dft_control_points -type async_set_reset`，
或 `add_nonscan_instances`

第二类：常数把复位一直打开

先查 test_setup 与已加的控制点，
确属功能意图才声明非扫描

第三类：回溯停下

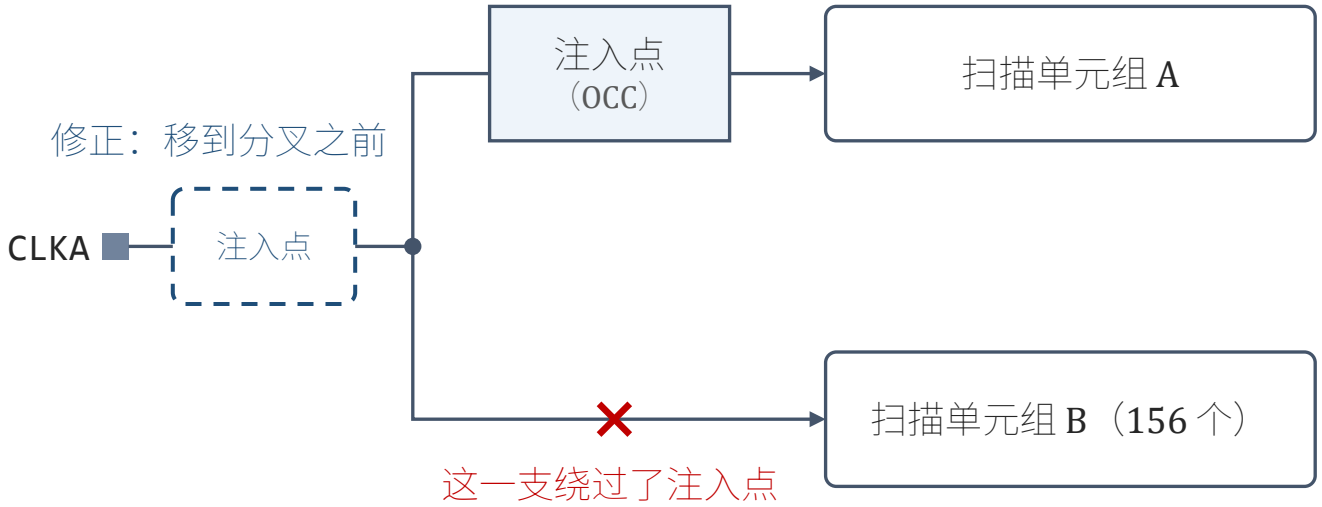
默认自动修：工具推断控制点，
用 `report_dft_control_points` 复核

第二、三类要先 `set_drc_handling DFT_C9 -auto_fix off` 才报出；默认 Error，可降为 Ignore

左下是第一类报告的模板，按本例填值；
OR 门移位时由 dynamic_disable = 1 关住复位

05：OCC 注入点要管住整个时钟源（DFT_C10）

注入点必须在时钟源的受控扇出里，时钟源驱动的扫描单元与存储器都要在它下面。



定义：新建时钟 `add_clocks -dft_inject_node`；已有时钟 `set_clock_options -dft_inject_node`

其余条件：不是常数；下游没有生成时钟的参考点；一个时钟源一个注入点（差分对除外）

等级：逻辑测试下默认 Warning，流程不停，但 OCC 管不住那一支

```
// Error: The clock 'CLKA' fanouts to 156 scannable elements which are
//        not the fanout of the specified -dft_inject_node
//        'tlb_inst1/CLK', … (DFT_C10-1)
```

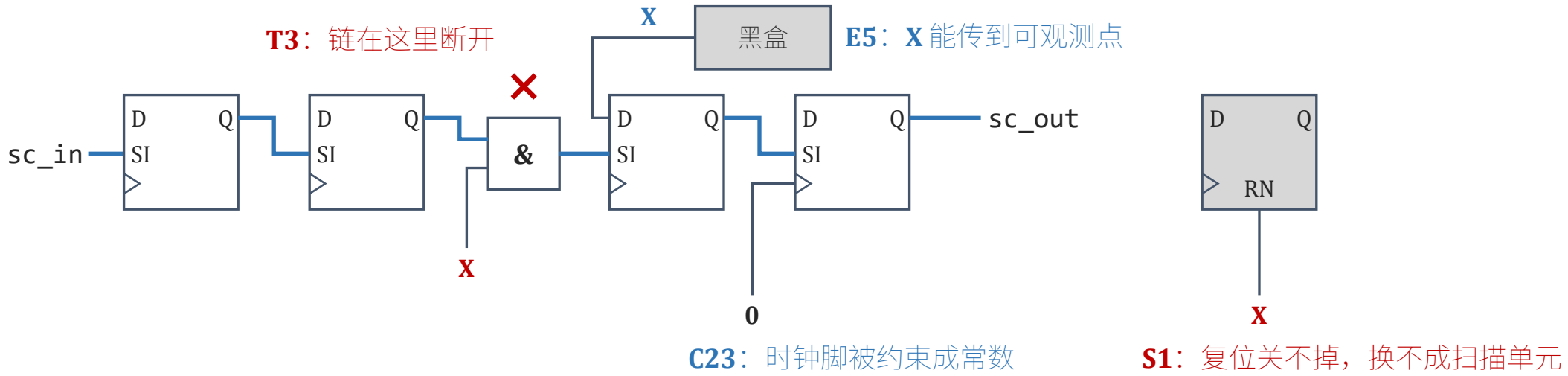
TCK 模式：DFT_C11–C13；缺信号时 `add_dft_signals tck_select`

OCC：On-Chip Clock controller；TCK 模式：存储器 BIST 用 TCK 作测试时钟的模式

→ 07、08 的报错看什么？

07、08：先看链通不通，再看哪些单元抓不到

后两步的报错少一些，但一出现就直接影响扫描链与覆盖率。



T3（**Error**，不能改）：先查测试过程与约束，再用 `analyze_drc_violation T3-1` 在原理图里沿 X 往回追

C23、**C25**（**Warning**）：流程不停，但覆盖率会掉；用 `set_gate_report -constrain_value on` 看常数从哪来

S1、**S2**（**Error**）：不过的触发器换不成扫描单元；**E5**、**D5**：增加 pattern 数、掉覆盖率

→ 报错之后去手册哪里查?

去哪查：规则号 → 手册那一节 → 命令与例子

前缀定位手册那一节；规则页给条件、报告、修法与示意图；命令页和 demo 给写法。

报告 **(DFT_C9-1)** → 前缀 **DFT_C** Pre-DFT Clock Rules → **Tessent Shell Reference Manual** 第 12 章 Design Rule Checking → 规则页 **DFT_C9**

- Default Handling
- 报告类型：Table 12-15
- 修法段落
- 示意图：Figure 12-38

→ 命令页 `add_dft_control_points`

→ demo 里的写法 `coreb_dft_rtl.dofile`

前缀 → 第 **12** 章的分类

DFT_C → Pre-DFT Clock	**DFT_S** → Pre-DFT Scan	**S** → Scannability	**T** → Scan Chain Trace
C → Clock	**D** → Scan Cell Data	**E** → Extra	**K** → EDT

`report_drc_rules DFT_C9` → `analyze_drc_violation DFT_C9-1` → 改设计、约束或测试过程 → `check_design_rules`

修复先改设计、约束或测试过程；工具自动加的控制点要复核；降级处理只作最后手段。

流程上下文：Tessent Shell User's Manual 的 RTL Design Rule Checks 一节；原理图也可在 Tessent Visualizer 的 DRC Browser 里看

感谢聆听

THANK YOU

BZ